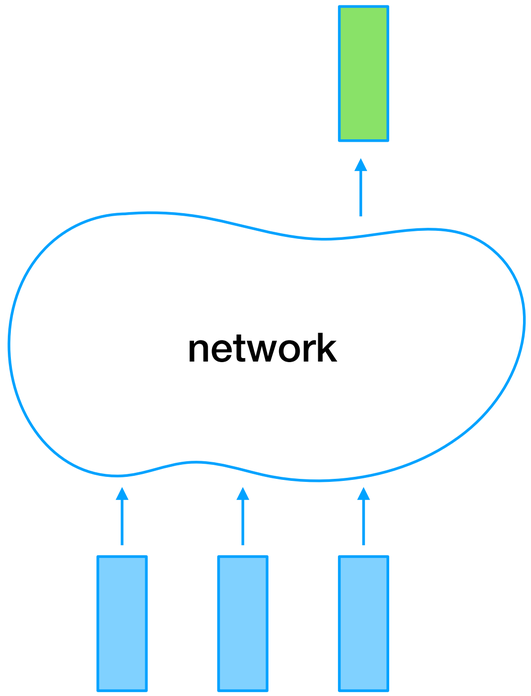
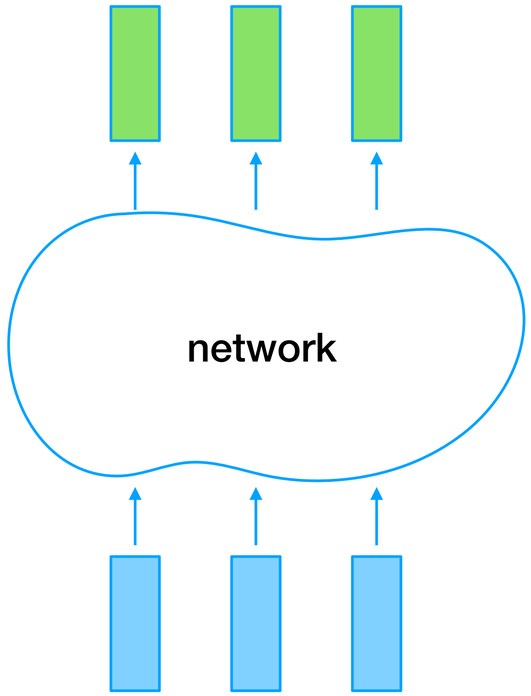
Recurrent networks

10-701 Introduction to Machine Learning
Geoff Gordon

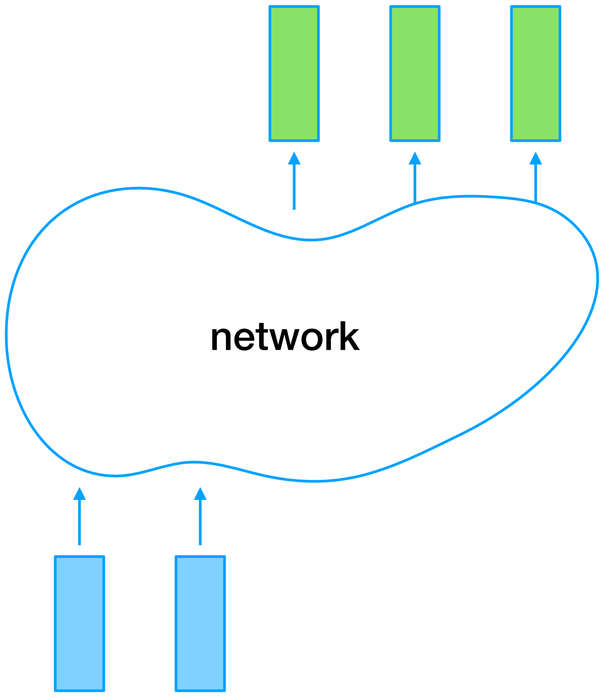
Sequence prediction tasks



many → one
classification, regression
(e.g., sentiment analysis)



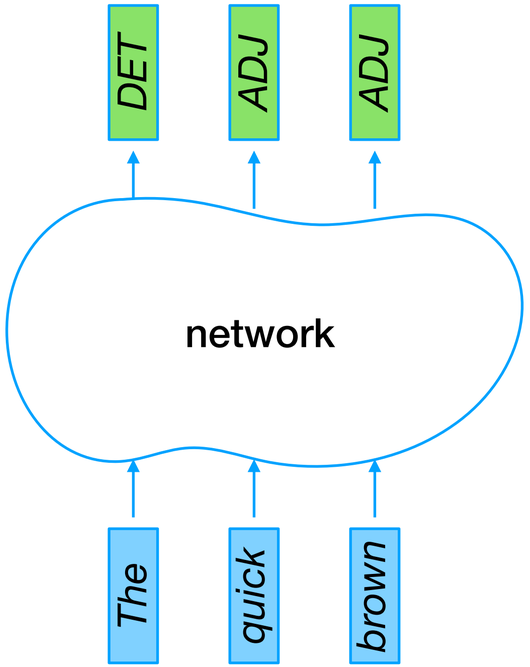
many → many (corresp.)
transduction
(e.g., POS tagging)



many → many
generation
(e.g., language model)

Part of speech tagging

- Example application: POS tagging
- Map each word (in context) to its part of speech (determiner, adjective, noun, verb, …)
- A transduction task: output tokens are matched with corresponding input tokens



OCR

another transduction task



- Could try to recognize each character independently, but accuracy will be higher if we use context

figure credit: (Chatzis & Demiris, 2013)

Masked token prediction

another transduction task



The quick brown fox jumped over the lazy dog

- For each token index i:
  - *input:* a window of tokens near index i but ***not*** token i: $j \in \{i-k, \ldots, i+k\} \setminus \{i\}$ (the ***context***)
  - *target:* the masked token i
- Could also mask more than 1 token in a context window

Masked token prediction

another transduction task



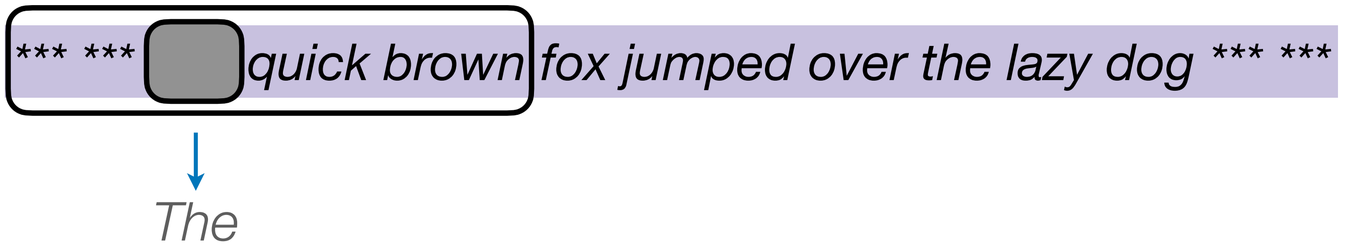
**** *** The quick brown fox jumped over the lazy dog *** ****

- For each token index i:
  - *input:* a window of tokens near index i but ***not*** token i: $j \in \{i-k, \ldots, i+k\} \setminus \{i\}$ (the ***context***)
  - *target:* the masked token i
- Could also mask more than 1 token in a context window

Masked token prediction

another transduction task





- For each token index i:
  - *input:* a window of tokens near index i but ***not*** token i: $j \in \{i-k, \ldots, i+k\} \setminus \{i\}$ (the ***context***)
  - *target:* the masked token i
- Could also mask more than 1 token in a context window

Masked token prediction

another transduction task





- For each token index i:
  - *input:* a window of tokens near index i but ***not*** token i: $j \in \{i-k, \ldots, i+k\} \setminus \{i\}$ (the ***context***)
  - *target:* the masked token i
- Could also mask more than 1 token in a context window

Masked token prediction

another transduction task





- For each token index i:
  - *input:* a window of tokens near index i but ***not*** token i: $j \in \{i-k, \ldots, i+k\} \setminus \{i\}$ (the ***context***)
  - *target:* the masked token i
- Could also mask more than 1 token in a context window

Masked token prediction

another transduction task



*** *** *The quick brown fox jumped over the lazy* [ ] *** ***

↓

dog

- For each token index i:
  - *input:* a window of tokens near index i but ***not*** token i: $j \in \{i-k, \ldots, i+k\} \setminus \{i\}$ (the ***context***)
  - *target:* the masked token i
- Could also mask more than 1 token in a context window

A generation task: language modeling

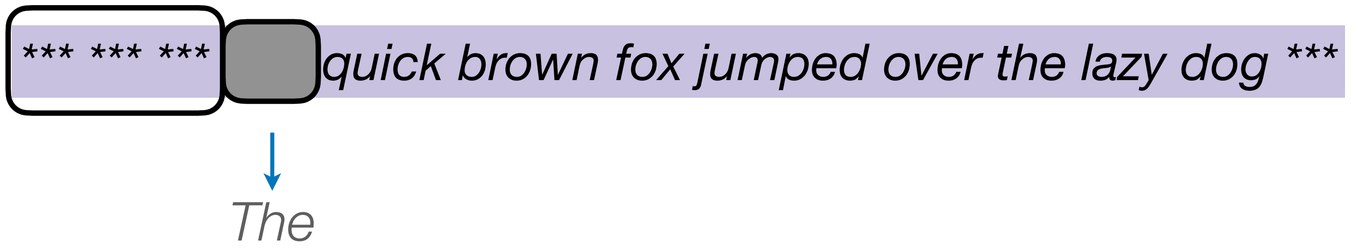
- Given a prompt (a string of tokens)
  - e.g., “3 × 17 – 9 = ” or “It was a dark and stormy night.”
- Predict a completion (another string of tokens)
  - e.g., “42” or “Suddenly, a shot rang out.”
  - goal: concatenation of prompt and completion should have high probability in training set distribution
  - i.e., max $P(\text{completion} \mid \text{prompt})$
- Often trained on the one-step version (just one token in the completion): ***next-token prediction***
  - we’ll talk more later about using next-token prediction repeatedly to generate a longer completion

Next token prediction

*** *** *** *The quick brown fox jumped over the lazy dog* ***

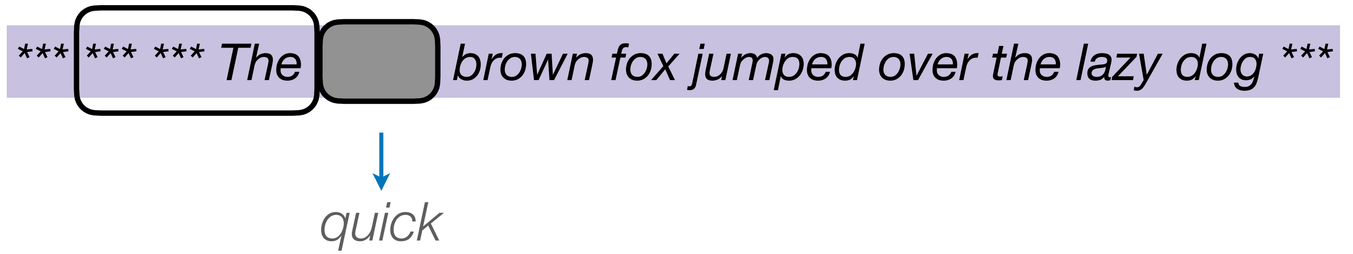
- For each token index i:
  - *input:* a window of tokens before index i but ***not*** token i: $j \in \{i-k, \ldots, i-1\}$ (the ***context***)
  - *target:* the next token i

Next token prediction



- For each token index i:
  - *input:* a window of tokens before index i but ***not*** token i: $j \in \{i-k, \ldots, i-1\}$ (the ***context***)
  - *target:* the next token i

Next token prediction



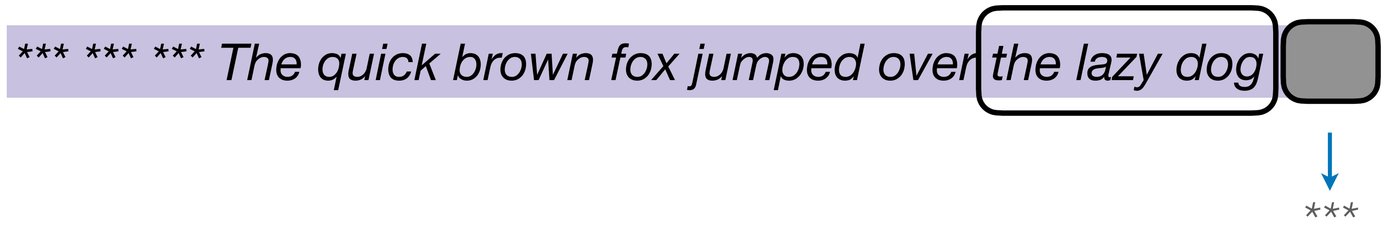
- For each token index i:
  - *input:* a window of tokens before index i but ***not*** token i: $j \in \{i-k, \ldots, i-1\}$ (the ***context***)
  - *target:* the next token i

Next token prediction



- For each token index i:
  - *input:* a window of tokens before index i but ***not*** token i: $j \in \{i-k, \ldots, i-1\}$ (the ***context***)
  - *target:* the next token i

Next token prediction



- For each token index i:
  - *input:* a window of tokens before index i but ***not*** token i: $j \in \{i-k, \ldots, i-1\}$ (the ***context***)
  - *target:* the next token i

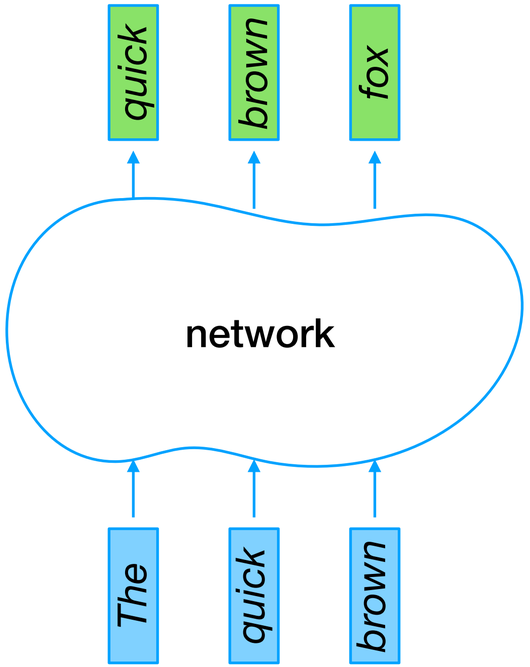
Recurrent neural network (RNN)

- Oldest sequence model
- Replicate a common network block for each step
  - input x_t, hidden state z_t, output y_t
- Hidden state at each step becomes another input to next step
  - initialize hidden state (e.g., to zero) before first step
  - weight matrix V connects steps, W gives embeddings

$$z_0 = 0$$

$$z_t = \sigma(Vz_{t-1} + Wx_t + b) \quad t = 1..T$$

note: no time index



*This is an **Elman network**, a type of **simple recurrent network**, applied to next-token prediction*

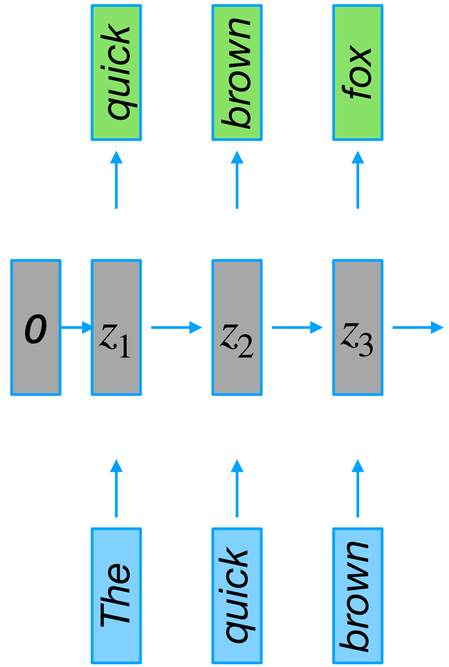
Recurrent neural network (RNN)

- Oldest sequence model
- Replicate a common network block for each step
  - input x_t, hidden state z_t, output y_t
- Hidden state at each step becomes another input to next step
  - initialize hidden state (e.g., to zero) before first step
  - weight matrix V connects steps, W gives embeddings

$$z_0 = 0$$

$$z_t = \sigma(Vz_{t-1} + Wx_t + b) \quad t = 1..T$$

note: no time index



*This is an **Elman network**, a type of **simple recurrent network**, applied to next-token prediction*

Example

- A tiny Elman RNN
  - task: see a sequence of tokens ABBABAB…, predict next token at each step
  - 2-dim hidden states, ReLU activation, cross-entropy loss

A tiny Elman network on {A, B}*

model and parameters

embed $e_t = E\,\text{onehot}(x_t)$

recur $z_t = Wh_{t-1} + e_t + b$

activate $h_t = \text{ReLU}(z_t)$

read out $o_t = Vh_t + c$

predict $p_t = \text{softmax}(o_t)$

loss $L = \sum_t -y_t \cdot \ln p_t, \quad y_t = \text{onehot}(x_{t+1})$

Parameters

E

$$\begin{bmatrix} 1.00 & -0.50 \\ -0.50 & 1.00 \end{bmatrix}$$

e_A e_B

W

$$\begin{bmatrix} 0.60 & -0.40 \\ 0.40 & 0.60 \end{bmatrix}$$

b

$$\begin{bmatrix} 0.30 \\ -0.20 \end{bmatrix}$$

V

$$\begin{matrix} \text{logit A} \\ \text{logit B} \end{matrix} \begin{bmatrix} 1.00 & 0.75 \\ -0.50 & -0.50 \end{bmatrix}$$

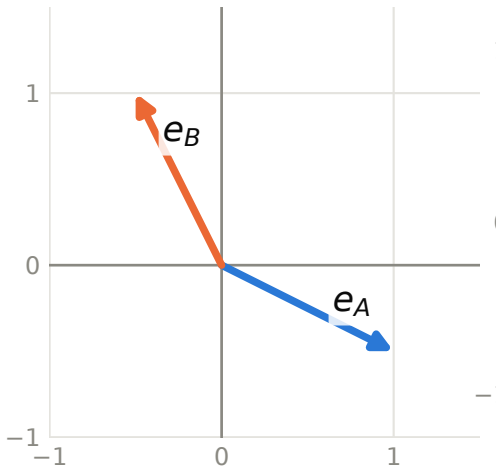
c

$$\begin{bmatrix} -2.00 \\ 0.00 \end{bmatrix}$$

$h_0 = (0,\ 0)$

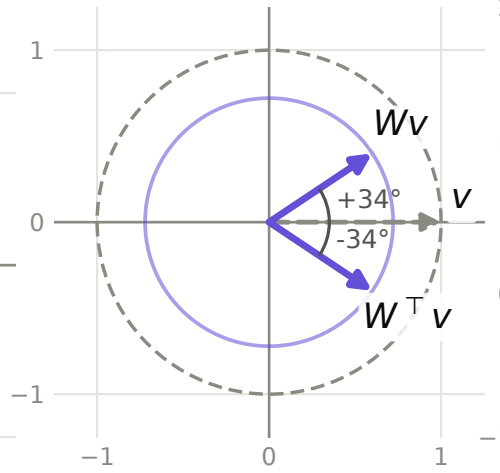
The columns of E are the embeddings of A and B; the rows of V give the logits for A and B. W rotates by 33.7° and scales by 0.72, so each step turns the old state a little and shrinks it.

Embeddings



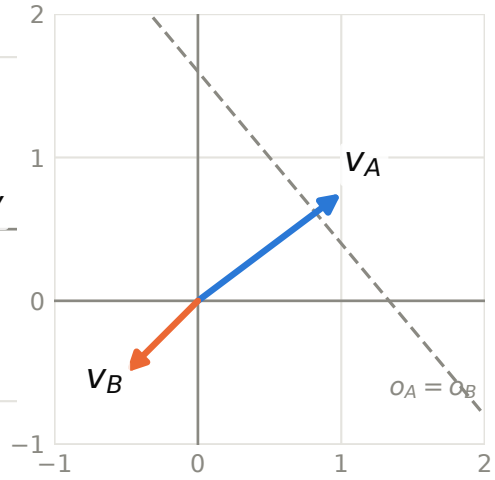
Columns of E: the vector added to the state when the network reads that token.

W and $W^\top$



W turns a vector counter-clockwise (forward pass). $W^\top$ turns it clockwise by the same angle (backward pass). Both shrink it by 0.72: the solid circle.

Read-out rows of V



Each logit is the state dotted with a row of V, plus c. Dashed: states where the two logits tie. The network predicts A above that line and B below it.

Forward pass, step 1 of 4

read x_1 = A, predict x_2

sequence **A** **A** B B A filled: input x_1 outlined: target x_2

1 embed

$e_1 = E\,\text{onehot}(x_1)$

$$\underset{E}{\begin{bmatrix}1.00 & -0.50\\ -0.50 & 1.00\end{bmatrix}}\ \underset{\text{onehot(A)}}{\begin{bmatrix}1\\0\end{bmatrix}} = \underset{e_A}{\begin{bmatrix}1.00\\-0.50\end{bmatrix}}$$

2 carry the state

Wh_0

the start state is zero

$$\underset{W}{\begin{bmatrix}0.60 & -0.40\\ 0.40 & 0.60\end{bmatrix}}\ \underset{h_0}{\begin{bmatrix}0.00\\0.00\end{bmatrix}} = \underset{Wh_0}{\begin{bmatrix}0.00\\0.00\end{bmatrix}}$$

3 add, then ReLU

$h_1 = \text{ReLU}(z_1)$

unit 2 is negative: clipped

$$\underset{Wh_0}{\begin{bmatrix}0.00\\0.00\end{bmatrix}} + \underset{e_A}{\begin{bmatrix}1.00\\-0.50\end{bmatrix}} + \underset{b}{\begin{bmatrix}0.30\\-0.20\end{bmatrix}} = \underset{z_1}{\begin{bmatrix}1.30\\-0.70\end{bmatrix}} \xrightarrow{\text{ReLU}} \underset{h_1}{\begin{bmatrix}1.30\\0.00\end{bmatrix}}$$

4 read out

$o_1 = Vh_1 + c$

$$\underset{V}{\begin{bmatrix}1.00 & 0.75\\ -0.50 & -0.50\end{bmatrix}}\ \underset{h_1}{\begin{bmatrix}1.30\\0.00\end{bmatrix}} + \underset{c}{\begin{bmatrix}-2.00\\0.00\end{bmatrix}} = \underset{o_1\ \text{(logits)}}{\begin{bmatrix}-0.70\\-0.65\end{bmatrix}}$$

5 predict

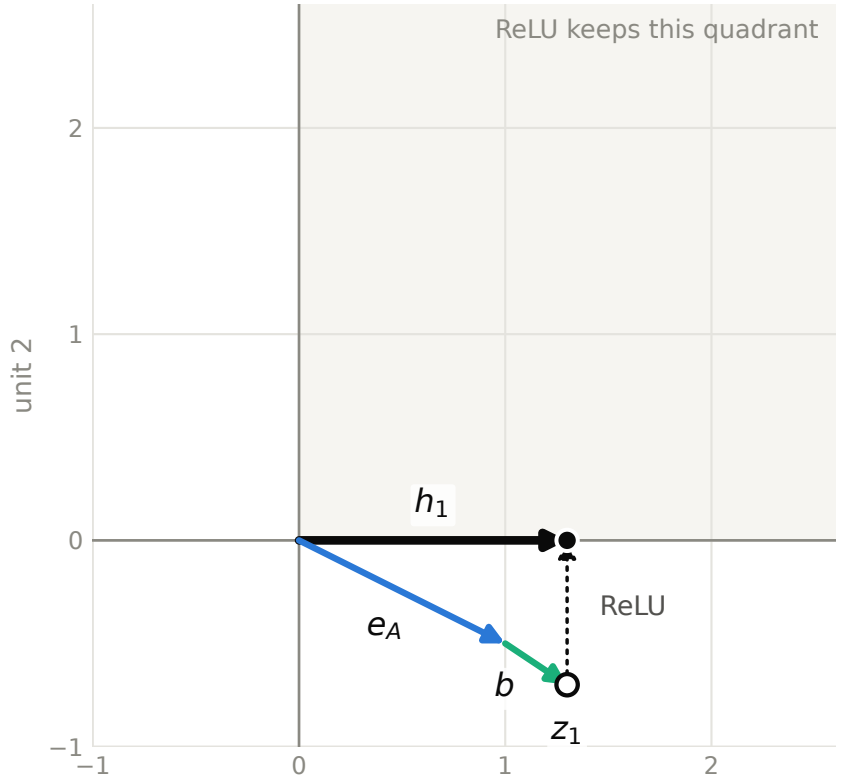
$p_1 = \text{softmax}(o_1)$

$$\text{softmax}\ \underset{o_1}{\begin{bmatrix}-0.70\\-0.65\end{bmatrix}} = \underset{p_1}{\begin{bmatrix}0.49\\0.51\end{bmatrix}}$$

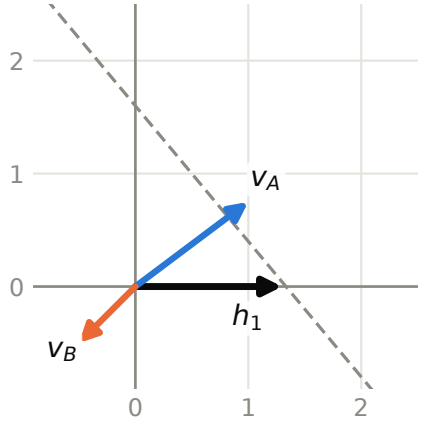
target y_1 = onehot(A) = (1, 0)

$L_1 = -y_1 \cdot \ln p_1 = -\ln 0.49 = 0.72$

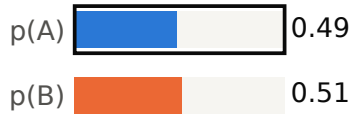
Hidden space



Read-out: h_1 against the rows of V



Prediction for x_2



state is below the dashed tie line → B more likely

Forward pass, step 2 of 4

read x_2 = A, predict x_3

sequence **A** **A** **B** **B** **A** filled: input x_2 outlined: target x_3

1 embed

$e_2 = E\,\text{onehot}(x_2)$

$$\underset{E}{\begin{bmatrix} 1.00 & -0.50 \\ -0.50 & 1.00 \end{bmatrix}} \underset{\text{onehot(A)}}{\begin{bmatrix} 1 \\ 0 \end{bmatrix}} = \underset{e_A}{\begin{bmatrix} 1.00 \\ -0.50 \end{bmatrix}}$$

2 carry the state

Wh_1

rotate and shrink

$$\underset{W}{\begin{bmatrix} 0.60 & -0.40 \\ 0.40 & 0.60 \end{bmatrix}} \underset{h_1}{\begin{bmatrix} 1.30 \\ 0.00 \end{bmatrix}} = \underset{Wh_1}{\begin{bmatrix} 0.78 \\ 0.52 \end{bmatrix}}$$

3 add, then ReLU

$h_2 = \text{ReLU}(z_2)$

unit 2 is negative: clipped

$$\underset{Wh_1}{\begin{bmatrix} 0.78 \\ 0.52 \end{bmatrix}} + \underset{e_A}{\begin{bmatrix} 1.00 \\ -0.50 \end{bmatrix}} + \underset{b}{\begin{bmatrix} 0.30 \\ -0.20 \end{bmatrix}} = \underset{z_2}{\begin{bmatrix} 2.08 \\ -0.18 \end{bmatrix}} \xrightarrow{\text{ReLU}} \underset{h_2}{\begin{bmatrix} 2.08 \\ 0.00 \end{bmatrix}}$$

4 read out

$o_2 = Vh_2 + c$

$$\underset{V}{\begin{bmatrix} 1.00 & 0.75 \\ -0.50 & -0.50 \end{bmatrix}} \underset{h_2}{\begin{bmatrix} 2.08 \\ 0.00 \end{bmatrix}} + \underset{c}{\begin{bmatrix} -2.00 \\ 0.00 \end{bmatrix}} = \underset{o_2\ \text{(logits)}}{\begin{bmatrix} 0.08 \\ -1.04 \end{bmatrix}}$$

5 predict

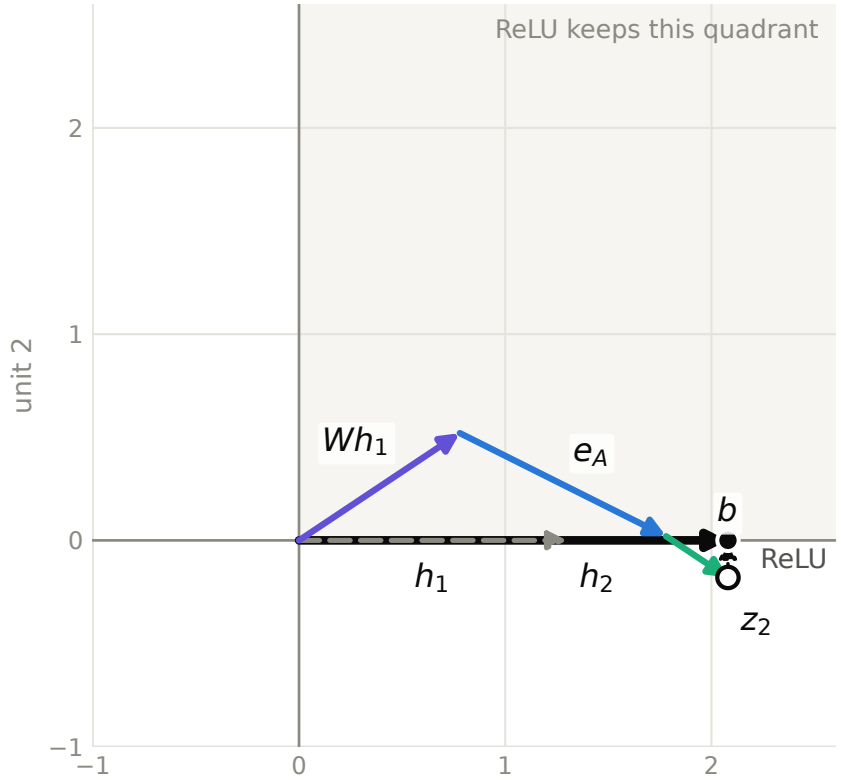
$p_2 = \text{softmax}(o_2)$

$$\text{softmax} \underset{o_2}{\begin{bmatrix} 0.08 \\ -1.04 \end{bmatrix}} = \underset{p_2}{\begin{bmatrix} 0.75 \\ 0.25 \end{bmatrix}}$$

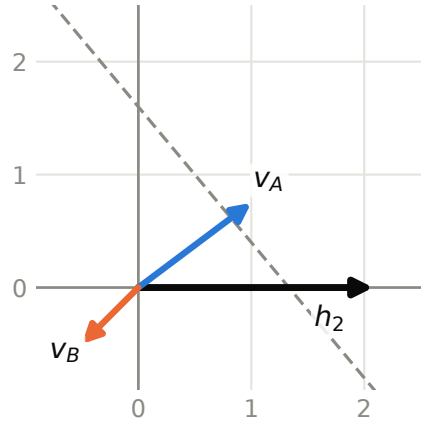
target y_2 = onehot(B) = (0, 1)

$L_2 = -y_2 \cdot \ln p_2 = -\ln 0.25 = 1.40$

Hidden space



Read-out: h_2 against the rows of V



Prediction for x_3



state is above the dashed tie line → A more likely

Forward pass, step 3 of 4

read x_3 = B, predict x_4

sequence **A** **A** **B** **B** **A** filled: input x_3 outlined: target x_4

1 embed

$e_3 = E\,\text{onehot}(x_3)$

$$\underset{E}{\begin{bmatrix}1.00 & -0.50\\ -0.50 & 1.00\end{bmatrix}}\ \underset{\text{onehot(B)}}{\begin{bmatrix}0\\1\end{bmatrix}} = \underset{e_B}{\begin{bmatrix}-0.50\\1.00\end{bmatrix}}$$

2 carry the state

Wh_2

rotate and shrink

$$\underset{W}{\begin{bmatrix}0.60 & -0.40\\ 0.40 & 0.60\end{bmatrix}}\ \underset{h_2}{\begin{bmatrix}2.08\\0.00\end{bmatrix}} = \underset{Wh_2}{\begin{bmatrix}1.25\\0.83\end{bmatrix}}$$

3 add, then ReLU

$h_3 = \text{ReLU}(z_3)$

both units stay active

$$\underset{Wh_2}{\begin{bmatrix}1.25\\0.83\end{bmatrix}} + \underset{e_B}{\begin{bmatrix}-0.50\\1.00\end{bmatrix}} + \underset{b}{\begin{bmatrix}0.30\\-0.20\end{bmatrix}} = \underset{z_3}{\begin{bmatrix}1.05\\1.63\end{bmatrix}} \xrightarrow{\text{ReLU}} \underset{h_3}{\begin{bmatrix}1.05\\1.63\end{bmatrix}}$$

4 read out

$o_3 = Vh_3 + c$

$$\underset{V}{\begin{bmatrix}1.00 & 0.75\\ -0.50 & -0.50\end{bmatrix}}\ \underset{h_3}{\begin{bmatrix}1.05\\1.63\end{bmatrix}} + \underset{c}{\begin{bmatrix}-2.00\\0.00\end{bmatrix}} = \underset{o_3\ \text{(logits)}}{\begin{bmatrix}0.27\\-1.34\end{bmatrix}}$$

5 predict

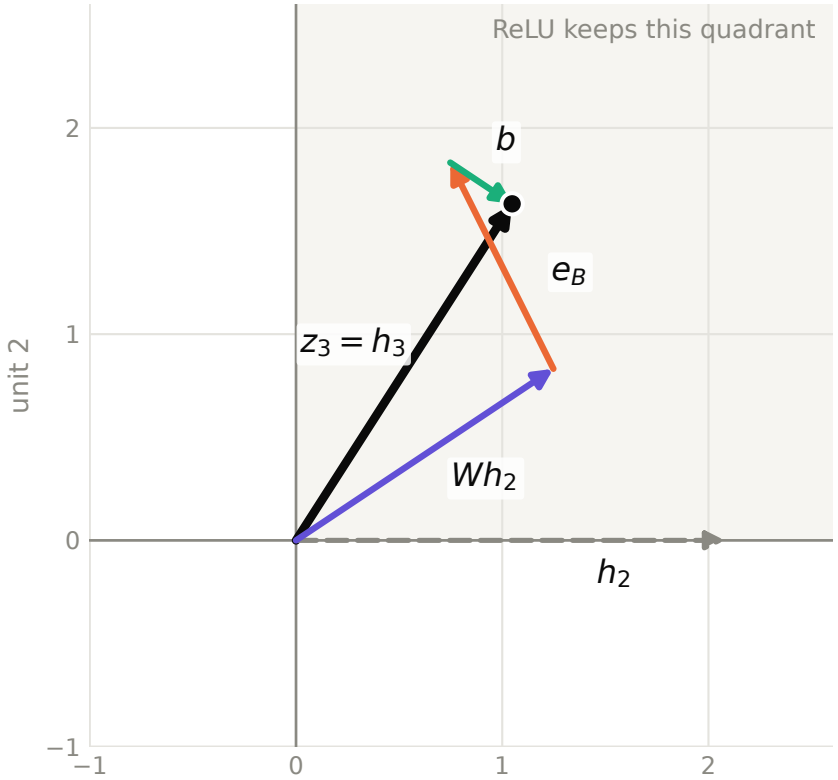
$p_3 = \text{softmax}(o_3)$

$$\text{softmax}\ \underset{o_3}{\begin{bmatrix}0.27\\-1.34\end{bmatrix}} = \underset{p_3}{\begin{bmatrix}0.83\\0.17\end{bmatrix}}$$

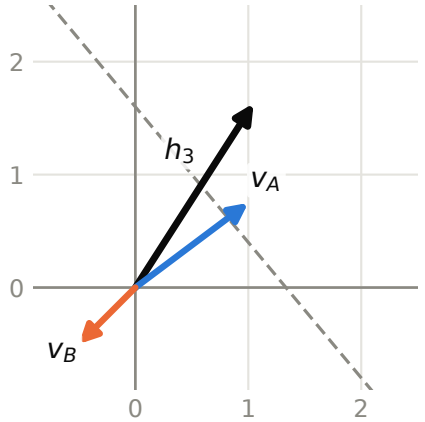
target y_3 = onehot(B) = (0, 1)

$L_3 = -y_3 \cdot \ln p_3 = -\ln 0.17 = 1.79$

Hidden space



Read-out: h_3 against the rows of V



Prediction for x_4



state is above the dashed tie line → A more likely

Forward pass, step 4 of 4

read x_4 = B, predict x_5

sequence **A** **A** **B** **B** **A** filled: input x_4 outlined: target x_5

1 embed

$e_4 = E\,\text{onehot}(x_4)$

$$\underset{E}{\begin{bmatrix}1.00 & -0.50\\ -0.50 & 1.00\end{bmatrix}}\ \underset{\text{onehot(B)}}{\begin{bmatrix}0\\1\end{bmatrix}} = \underset{e_B}{\begin{bmatrix}-0.50\\1.00\end{bmatrix}}$$

2 carry the state

Wh_3

rotate and shrink

$$\underset{W}{\begin{bmatrix}0.60 & -0.40\\ 0.40 & 0.60\end{bmatrix}}\ \underset{h_3}{\begin{bmatrix}1.05\\1.63\end{bmatrix}} = \underset{Wh_3}{\begin{bmatrix}-0.02\\1.40\end{bmatrix}}$$

3 add, then ReLU

$h_4 = \text{ReLU}(z_4)$

unit 1 is negative: clipped

$$\underset{Wh_3}{\begin{bmatrix}-0.02\\1.40\end{bmatrix}} + \underset{e_B}{\begin{bmatrix}-0.50\\1.00\end{bmatrix}} + \underset{b}{\begin{bmatrix}0.30\\-0.20\end{bmatrix}} = \underset{z_4}{\begin{bmatrix}-0.22\\2.20\end{bmatrix}} \xrightarrow{\text{ReLU}} \underset{h_4}{\begin{bmatrix}0.00\\2.20\end{bmatrix}}$$

4 read out

$o_4 = Vh_4 + c$

$$\underset{V}{\begin{bmatrix}1.00 & 0.75\\ -0.50 & -0.50\end{bmatrix}}\ \underset{h_4}{\begin{bmatrix}0.00\\2.20\end{bmatrix}} + \underset{c}{\begin{bmatrix}-2.00\\0.00\end{bmatrix}} = \underset{o_4\ \text{(logits)}}{\begin{bmatrix}-0.35\\-1.10\end{bmatrix}}$$

5 predict

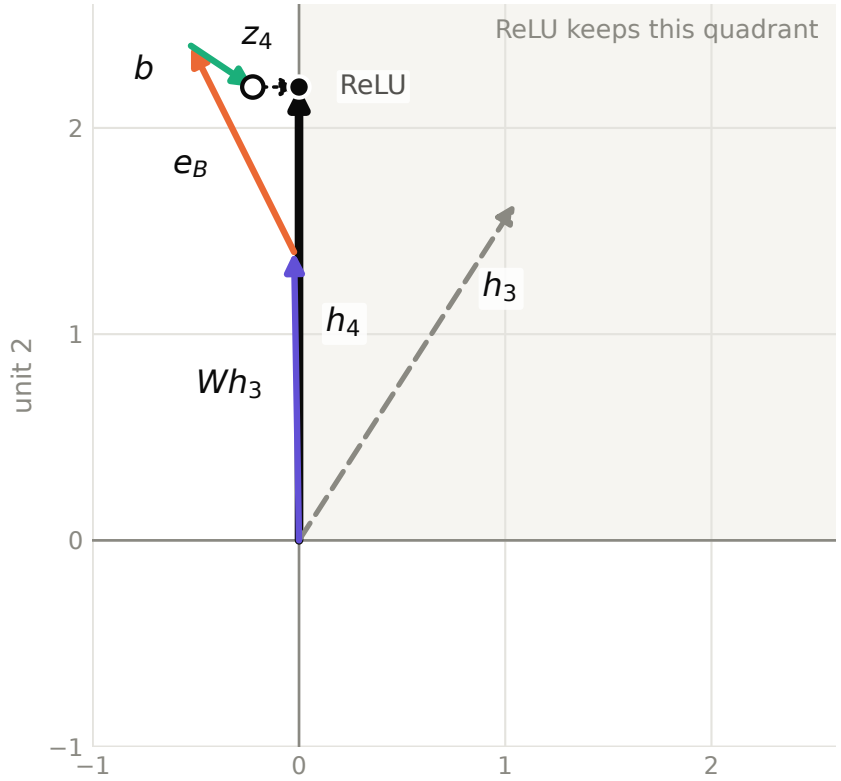
$p_4 = \text{softmax}(o_4)$

$$\text{softmax}\ \underset{o_4}{\begin{bmatrix}-0.35\\-1.10\end{bmatrix}} = \underset{p_4}{\begin{bmatrix}0.68\\0.32\end{bmatrix}}$$

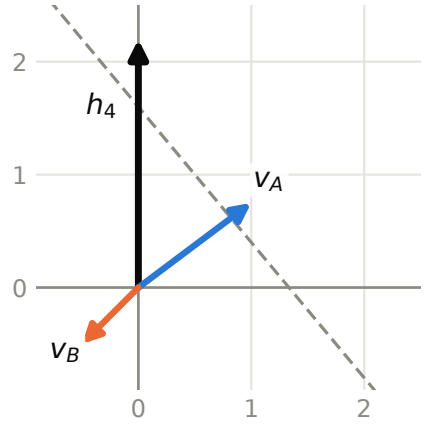
target y_4 = onehot(A) = (1, 0)

$L_4 = -y_4 \cdot \ln p_4 = -\ln 0.68 = 0.39$

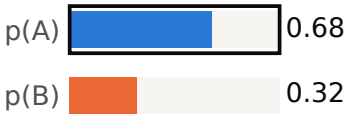
Hidden space



Read-out: h_4 against the rows of V



Prediction for x_5



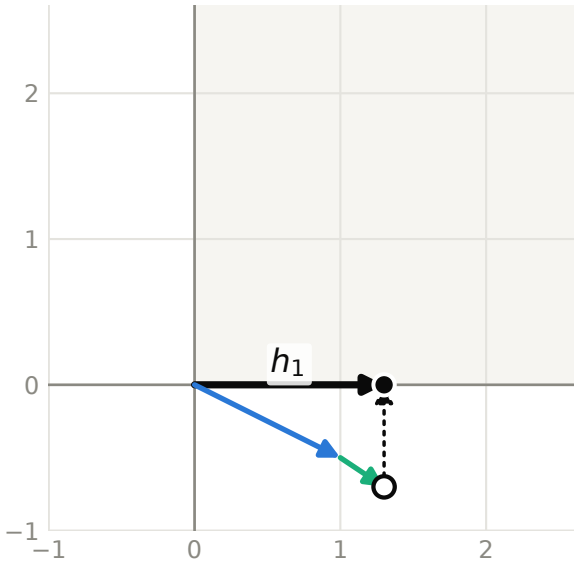
state is above the dashed tie line → A more likely

Forward pass, unrolled

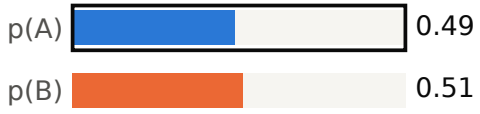
total loss L = 4.30

previous state — Wh_{t-1} — $+e_A$ — $+e_B$ — $+b$ — h_t (after ReLU)

t = 1 input A

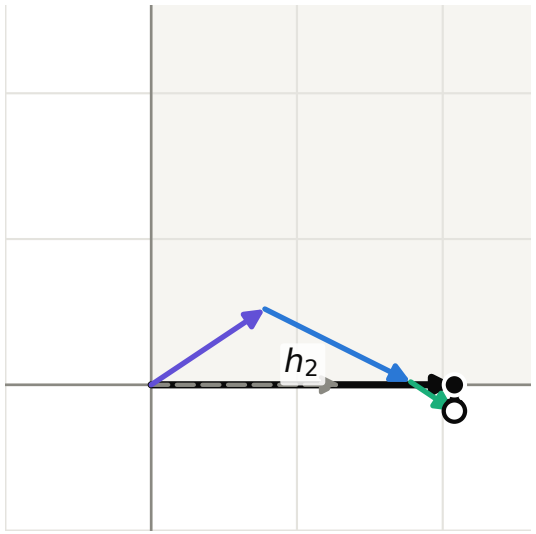


h_1 = (1.30, 0.00)



target A L_1 = 0.72

t = 2 input A

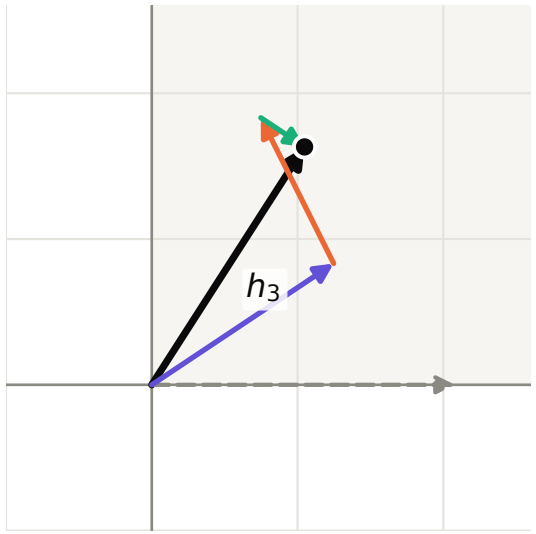


h_2 = (2.08, 0.00)



target B L_2 = 1.40

t = 3 input B

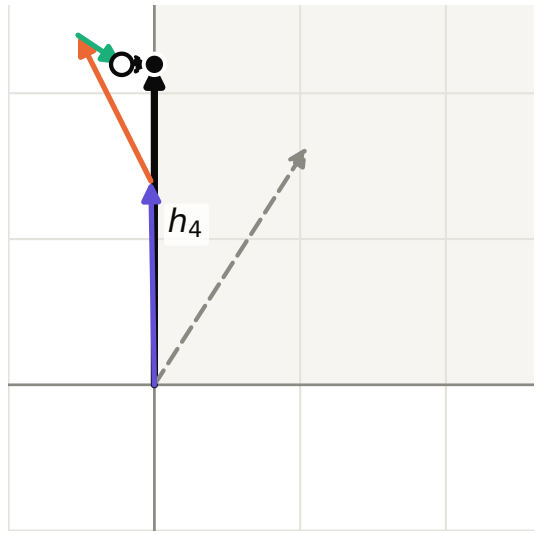


h_3 = (1.05, 1.63)



target B L_3 = 1.79

t = 4 input B



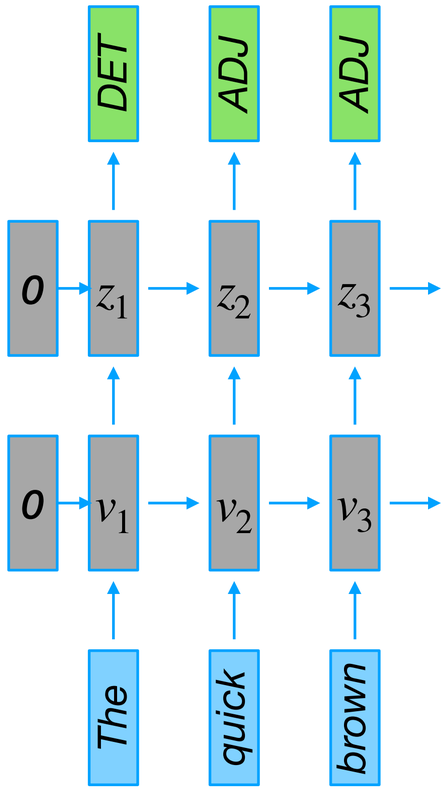
h_4 = (0.00, 2.20)



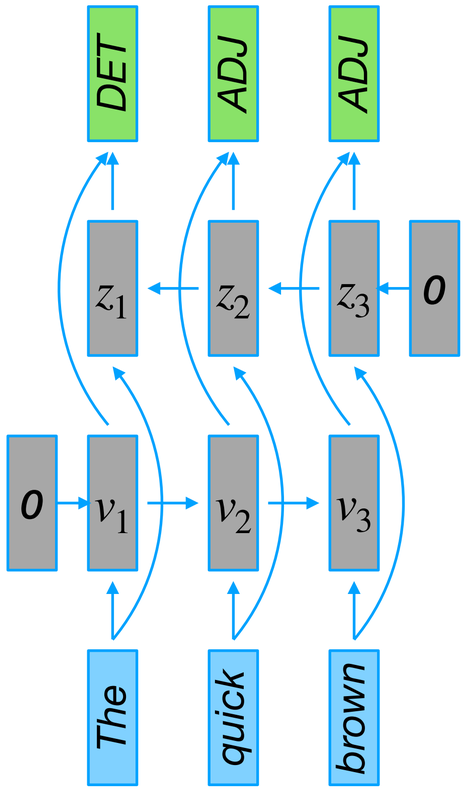
target A L_4 = 0.39

Variants

shown here on POS tagging (transduction)



stacked



bidirectional

- Lots of variants of RNNs — incl. multiple kinds of memory, see below

Training RNNs

note: vanishing/ exploding gradients!

(same tools as before + one more below)

- Once we unroll it, RNN is just like any other network
  - forward pass from $t = 1..T$
  - backward pass from $t = T..1$ (*backprop through time)*
  - SGD step
- Similar to deep feedforward net (MLP)
  - difference #1: same parameters V, W, b at every step
    - autodiff handles parameter sharing automatically
    - or we can manually sum gradients (total derivative rule)
  - difference #2: multiple output nodes
    - loss = sum(losses at each output), so gradients backprop from each output and sum at every parameter
    - have to run all time steps before we can backprop

Backward pass, step 4 of 4

last step: nothing arrives from the future

sequence A A B B A filled: input x_4 outlined: target x_5

gradient at the probabilities
$\partial L_4/\partial p_4 = -y_4 / p_4$
[-1.47, 0.00]
$L_4 = -y_4 \cdot \ln p_4$
$y_4 = (1, 0)$, $p_4 = (0.68, 0.32)$

gradient at the logits
$\partial L_4/\partial o_4 = p_4 - y_4$
[-0.32, 0.32]
back through softmax:
$(\mathrm{diag}(p) - pp^\top)(-y/p) = p - y$

from this step's loss
$V^\top (p_4 - y_4)$
[-0.48, -0.40]
a combination of the rows of V:
$-0.32\, v_A + 0.32\, v_B$

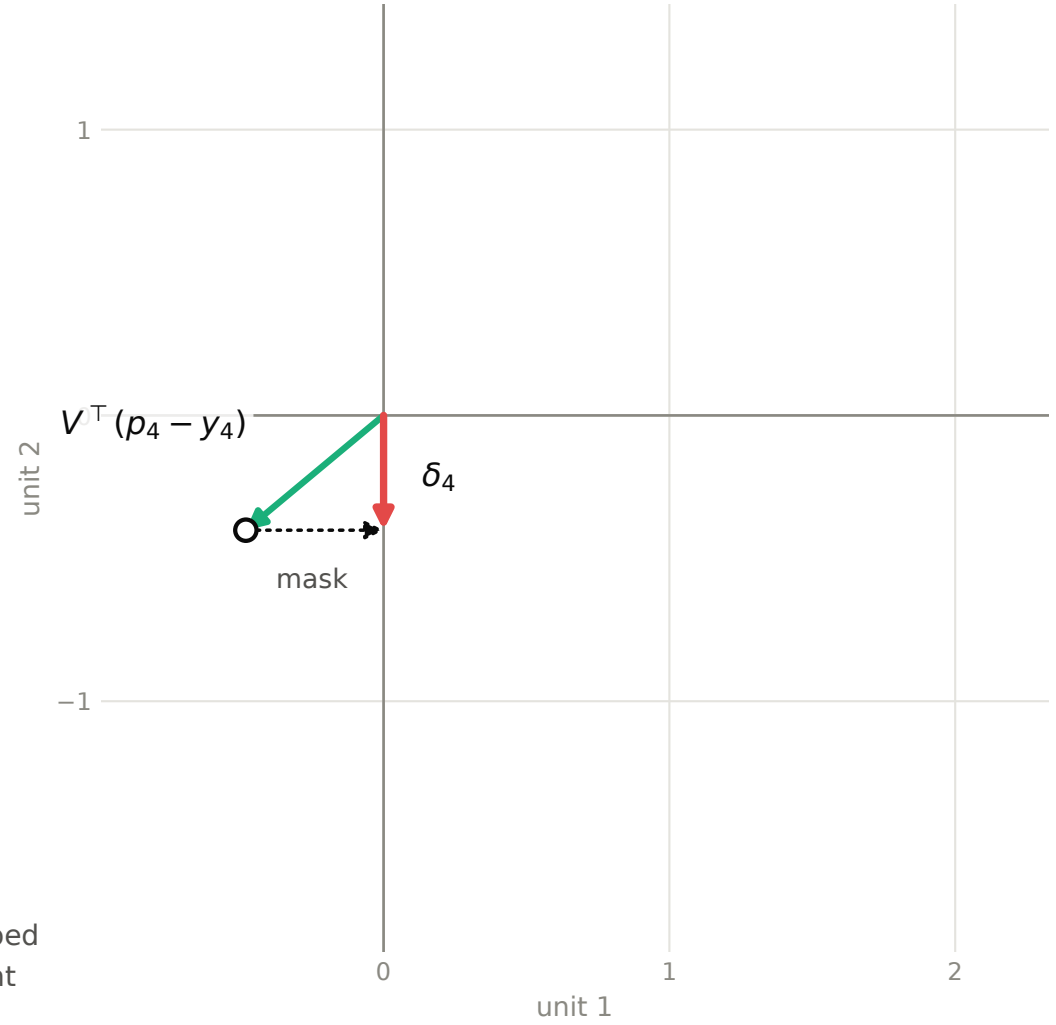
from the future
$W^\top \delta_5$
[0.00, 0.00]
no later step

gradient at the state
$g_4 = \partial L/\partial h_4$ (sum)
[-0.48, -0.40]

ReLU mask
$[z_4 > 0]$
[0.00, 1.00]
z_4 = (-0.22, 2.20)

gradient before ReLU
$\delta_4 = g_4 \odot \mathrm{mask}$
[0.00, -0.40]
unit 1 was clipped going forward: zeroed
unit 2 was active: its negative gradient passes (the mask looks at z, not at g)

Gradient vectors in hidden space



δ_4 is also this step's gradient for b and for e_B, and is sent on to step 3.

Backward pass, step 3 of 4

gradient arrives from step 4 through $W^\top$

sequence **A** **A** **B** **B** **A** filled: input x_3 outlined: target x_4

gradient at the probabilities
$\partial L_3/\partial p_3 = -y_3 / p_3$
[0.00, -6.01]
$L_3 = -y_3 \cdot \ln p_3$
y_3 = (0, 1), p_3 = (0.83, 0.17)

gradient at the logits
$\partial L_3/\partial o_3 = p_3 - y_3$
[0.83, -0.83]
back through softmax:
$(\mathrm{diag}(p) - pp^\top)(-y/p) = p - y$

from this step's loss
$V^\top (p_3 - y_3)$
[1.25, 1.04]
a combination of the rows of V:
$+0.83\, v_A \; - 0.83\, v_B$

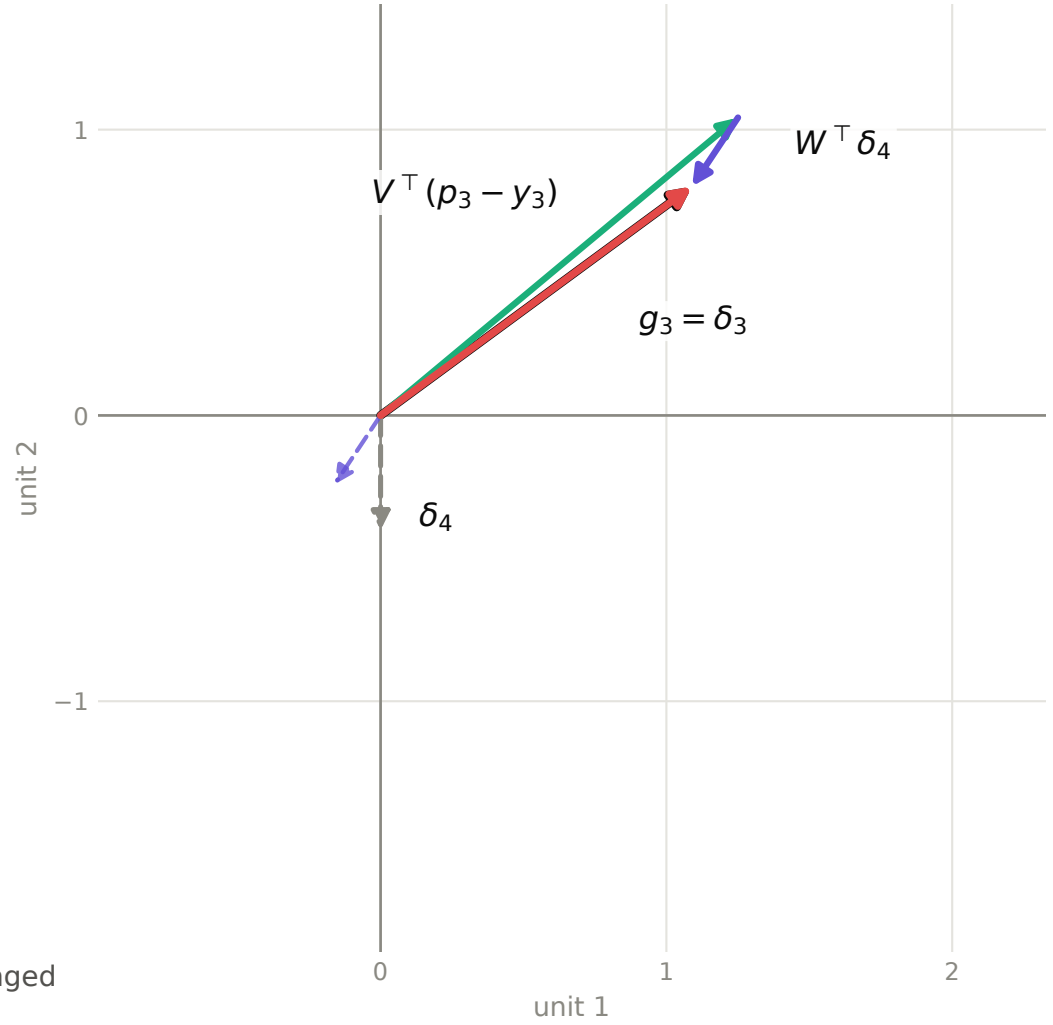
from the future
$W^\top \delta_4$
[-0.16, -0.24]
δ_4 = (0.00, -0.40)
rotated back and shrunk

gradient at the state
$g_3 = \partial L/\partial h_3$ (sum)
[1.09, 0.80]

ReLU mask
$[z_3 > 0]$
[1.00, 1.00]
z_3 = (1.05, 1.63)

gradient before ReLU
$\delta_3 = g_3 \odot \mathrm{mask}$
[1.09, 0.80]
both units were active: passes unchanged

Gradient vectors in hidden space



δ_3 is also this step's gradient for b and for e_B, and is sent on to step 2.

Backward pass, step 2 of 4

gradient arrives from step 3 through $W^\top$

sequence **A** **A** **B** **B** **A** filled: input x_2 outlined: target x_3

gradient at the probabilities
$\partial L_2/\partial p_2 = -y_2 / p_2$
0.00
-4.06
$L_2 = -y_2 \cdot \ln p_2$
y_2 = (0, 1), p_2 = (0.75, 0.25)

gradient at the logits
$\partial L_2/\partial o_2 = p_2 - y_2$
0.75
-0.75
back through softmax:
$(\mathrm{diag}(p) - pp^\top)(-y/p) = p - y$

from this step's loss
$V^\top (p_2 - y_2)$
1.13
0.94
a combination of the rows of V:
$+0.75\, v_A \; - 0.75\, v_B$

from the future
$W^\top \delta_3$
0.97
0.04
δ_3 = (1.09, 0.80)
rotated back and shrunk

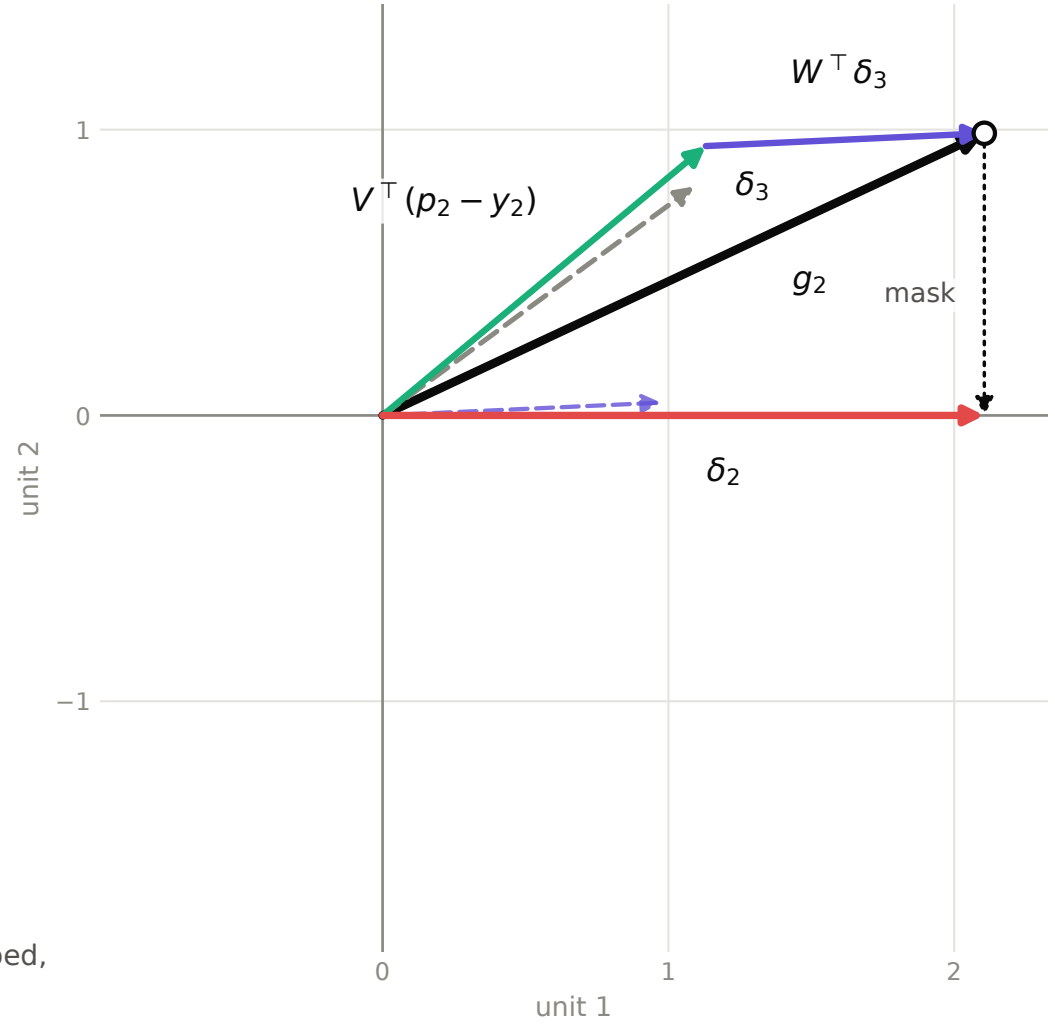
gradient at the state
$g_2 = \partial L/\partial h_2$ (sum)
2.11
0.99

ReLU mask
$[z_2 > 0]$
1.00
0.00
z_2 = (2.08, -0.18)

gradient before ReLU
$\delta_2 = g_2 \odot \mathrm{mask}$
2.11
0.00
unit 2 was clipped going forward: zeroed, even though its gradient is positive

δ_2 is also this step's gradient for b and for e_A, and is sent on to step 1.

Gradient vectors in hidden space



Backward pass, step 1 of 4

gradient arrives from step 2 through $W^\top$

sequence **A** **A** B B A filled: input x_1 outlined: target x_2

gradient at the probabilities
$\partial L_1/\partial p_1 = -y_1 / p_1$
[-2.05, 0.00]
$L_1 = -y_1 \cdot \ln p_1$
$y_1 = (1, 0)$, $p_1 = (0.49, 0.51)$

gradient at the logits
$\partial L_1/\partial o_1 = p_1 - y_1$
[-0.51, 0.51]
back through softmax:
$(\mathrm{diag}(p) - pp^\top)(-y/p) = p - y$

from this step's loss
$V^\top (p_1 - y_1)$
[-0.77, -0.64]
a combination of the rows of V:
$-0.51\, v_A + 0.51\, v_B$

from the future
$W^\top \delta_2$
[1.26, -0.84]
$\delta_2 = (2.11, 0.00)$
rotated back and shrunk

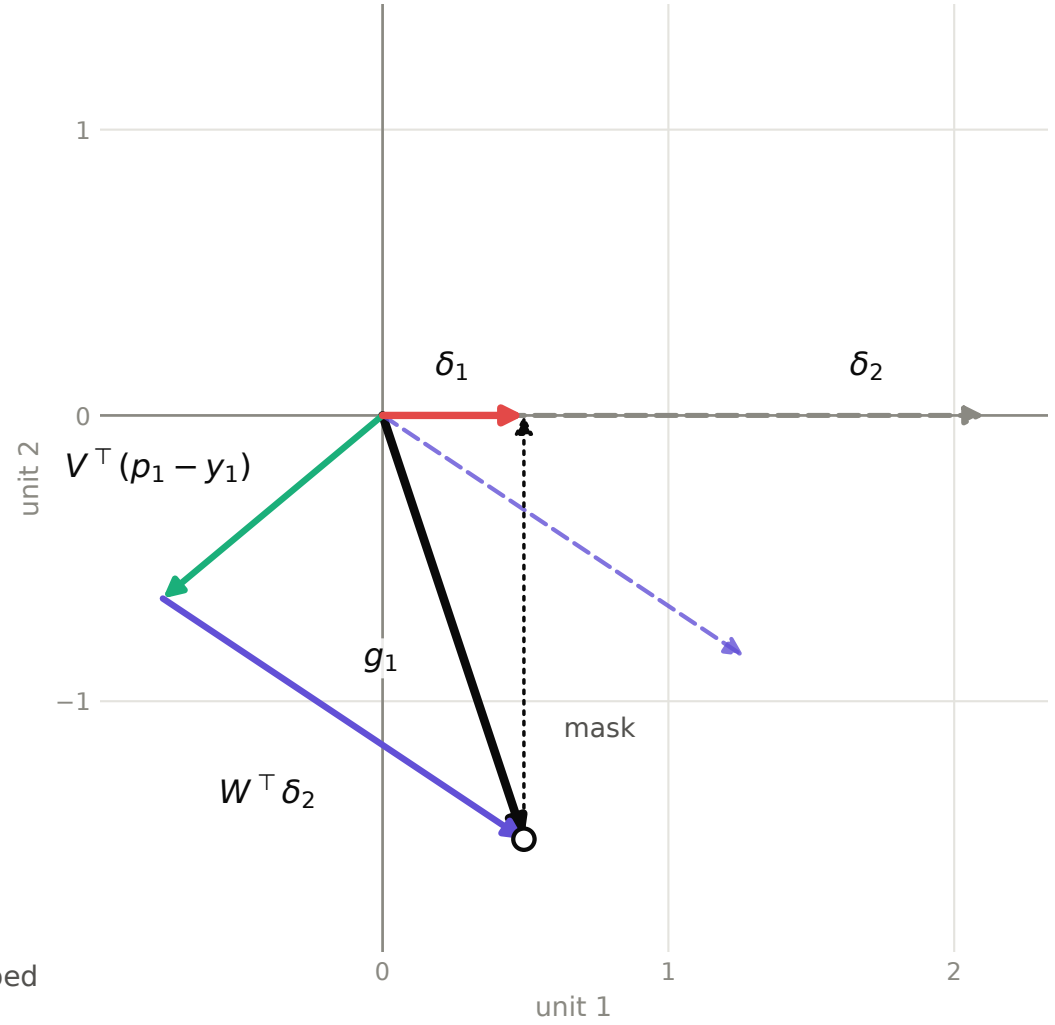
gradient at the state
$g_1 = \partial L/\partial h_1$ (sum)
[0.49, -1.48]

ReLU mask
$[z_1 > 0]$
[1.00, 0.00]
$z_1 = (1.30, -0.70)$

gradient before ReLU
$\delta_1 = g_1 \odot \mathrm{mask}$
[0.49, 0.00]
unit 2 was clipped going forward: zeroed

δ_1 is also this step's gradient for b and for e_A.

Gradient vectors in hidden space

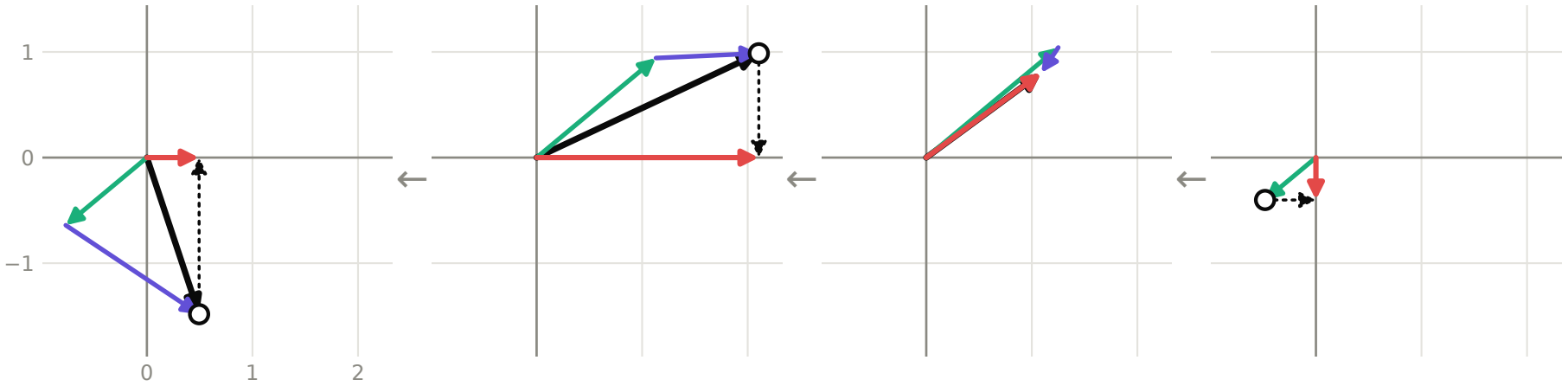


Backward pass, unrolled

read right to left

from this step's loss — from step $t+1$ (via $W^\top$) — sum g_t — δ_t after the ReLU mask

t = 1 input **A** | **t = 2** input **A** | **t = 3** input **B** | **t = 4** input **B**



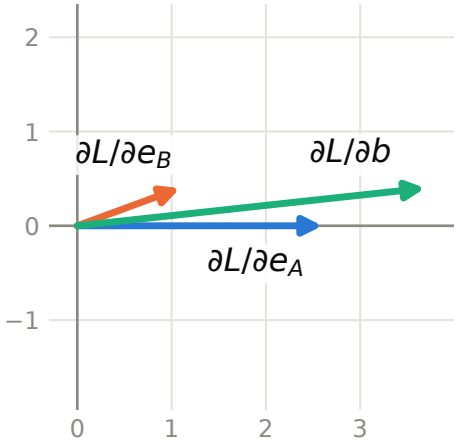
δ_1 = (0.49, 0.00)
mask (1, 0)

δ_2 = (2.11, 0.00)
mask (1, 0)

δ_3 = (1.09, 0.80)
mask (1, 1)

δ_4 = (0.00, -0.40)
mask (0, 1)

Parameter gradients



Each embedding collects δ from the steps where its token was read; the bias collects δ from every step.

$\partial L/\partial e_A = \delta_1 + \delta_2$ = (2.60, 0.00)

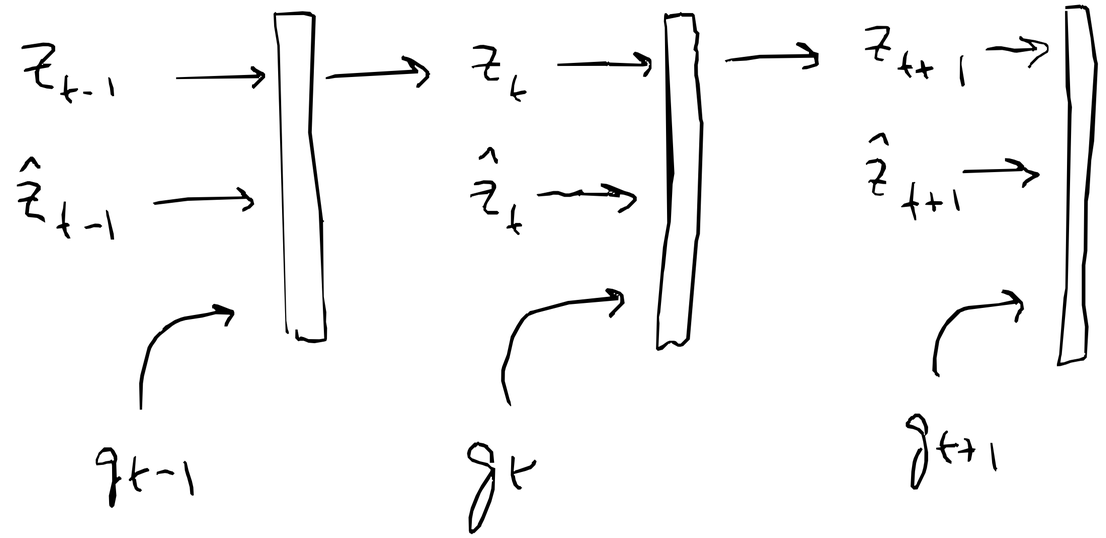
$\partial L/\partial e_B = \delta_3 + \delta_4$ = (1.09, 0.40)

$\partial L/\partial b = \sum_t \delta_t$ = (3.69, 0.40)

A gradient-descent step moves each parameter opposite to its arrow.

Memory

Unlike residual connection, to couple long distances, need to train gate to be near 1 (don't replace) in between



minimal gated unit (MGU)

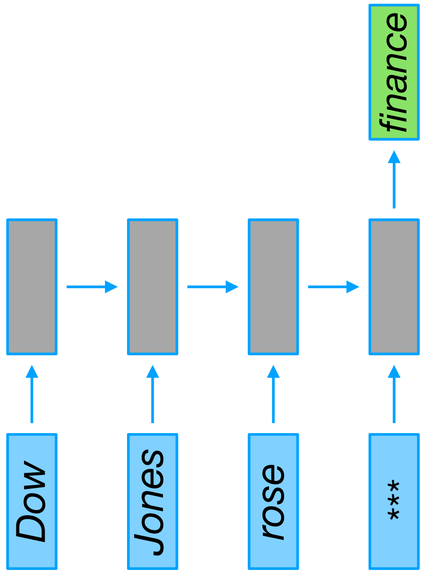
gating: $z_{t+1} = g_t \circ z_t + (1 - g_t) \circ \hat{z}_t$

both gate g_t *and candidate update* $\hat{z}_t$ *are of form* $f(Vz_{t-1} + Wx_t + b)$

output y_t *could depend on* z_t, x_t

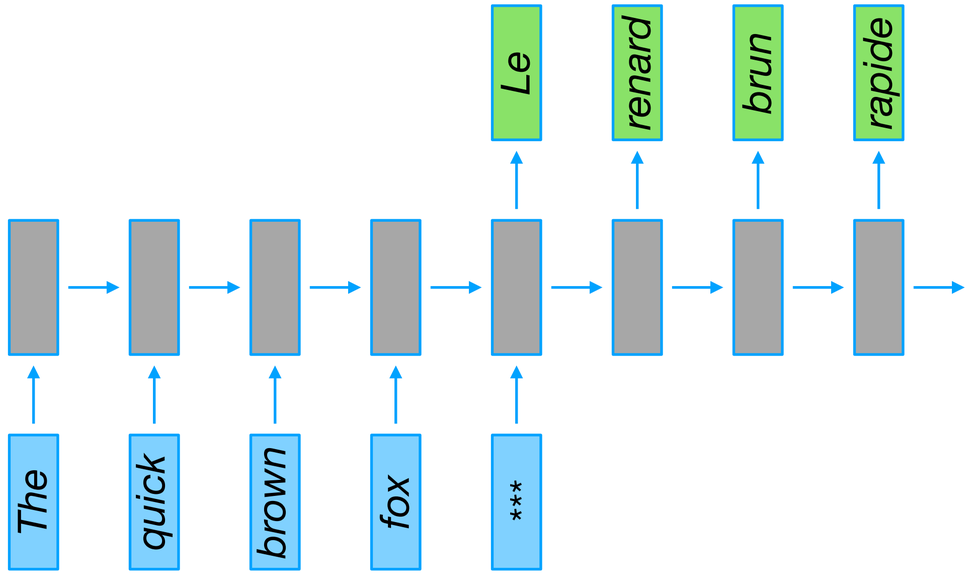
- Use *gating* to update hidden state
  - differentiable decision to store into z_t
  - like a controllable residual connection that lets us connect storage to retrieval
- Examples: LSTM, GRU, MGU

RNN for many:1 tasks



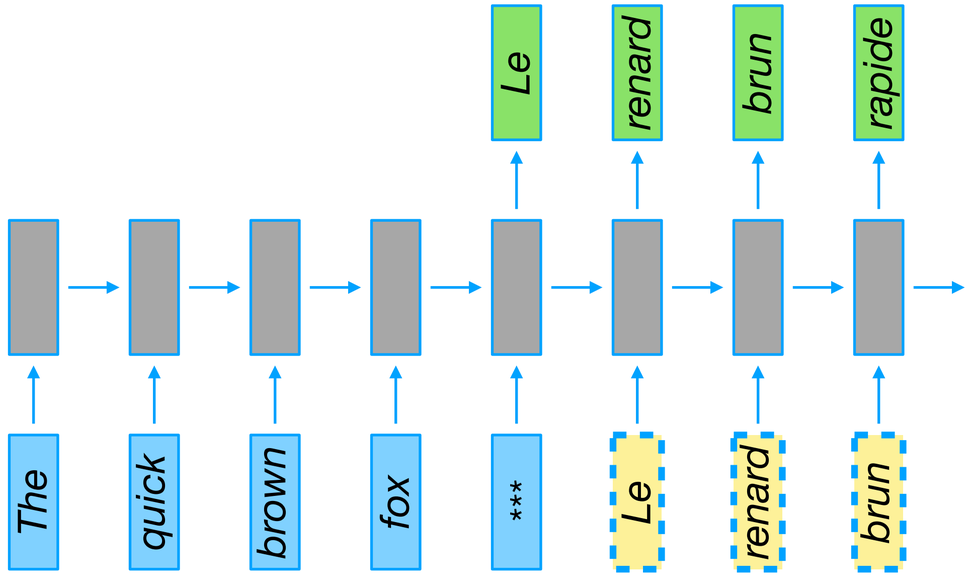
- To handle many:1 tasks like classification, most common setup is to end with a special token like ***, predict label based on corresponding hidden vector

RNN for generation



- To generate an arbitrary-length output, we can use RNN as a generative model
- E.g., language model: process input tokens (prompt), mark end of prompt w/ ***, continue after *** with next-token prediction
  - each time step generates one new token; stop when we generate *** again

RNN for generation

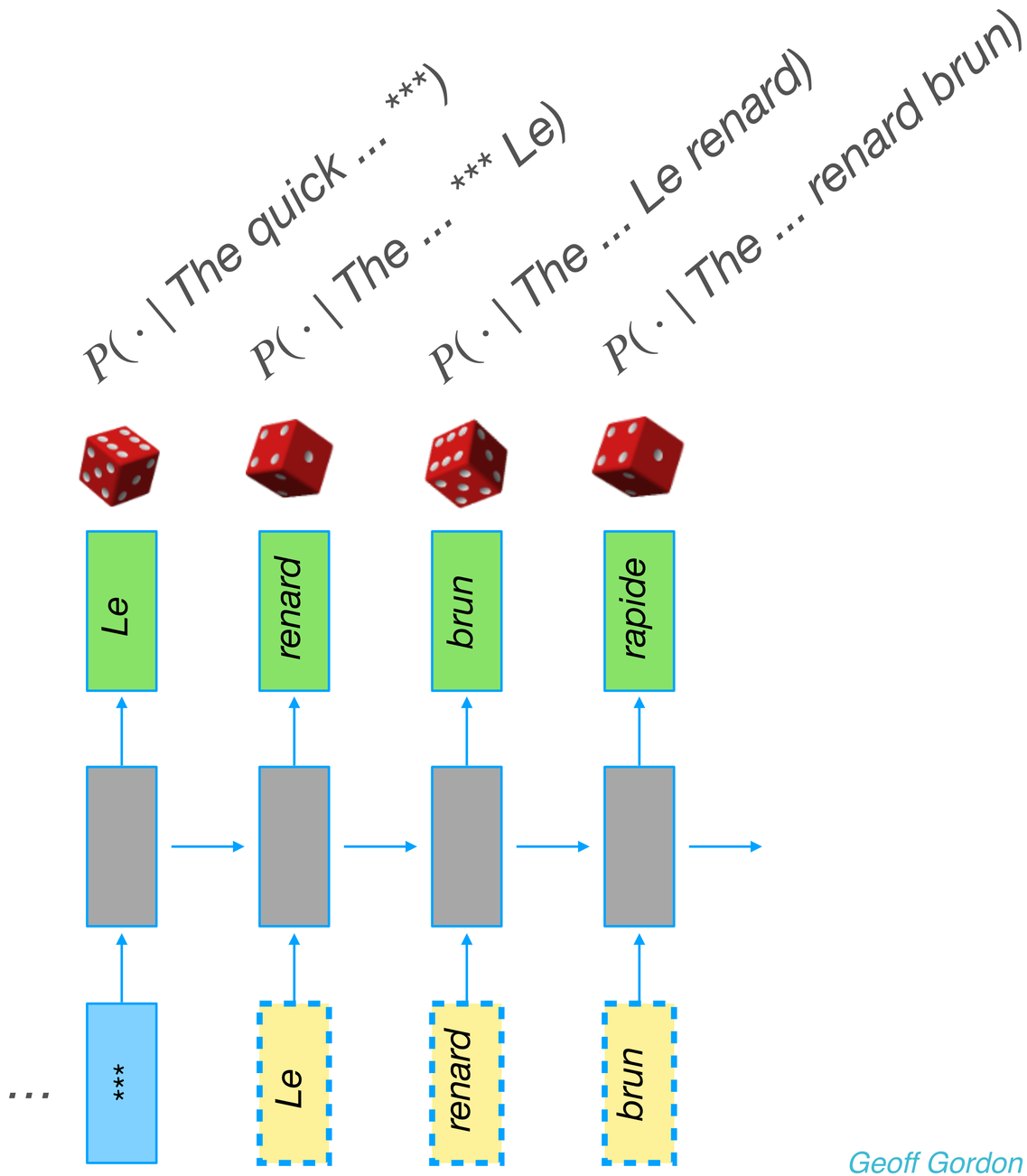


- To generate an arbitrary-length output, we can use RNN as a generative model
- E.g., language model: process input tokens (prompt), mark end of prompt w/ ***, continue after *** with next-token prediction
  - each time step generates one new token; stop when we generate *** again

$$P(\text{Le renard brun rapide} \mid \text{prompt}) = P(\text{Le} \mid \text{prompt}) \cdot P(\text{renard} \mid \text{prompt}, \text{Le}) \cdot \ldots$$

Generation process (sequence completion)

- Output token at each step is a sample from *conditional probability distribution* given preceding tokens
- Like a weighted die with 50k sides
  - weights are model output
  - roll, pick the token that lands face up
- Copy this token to input (condition on it) to generate next



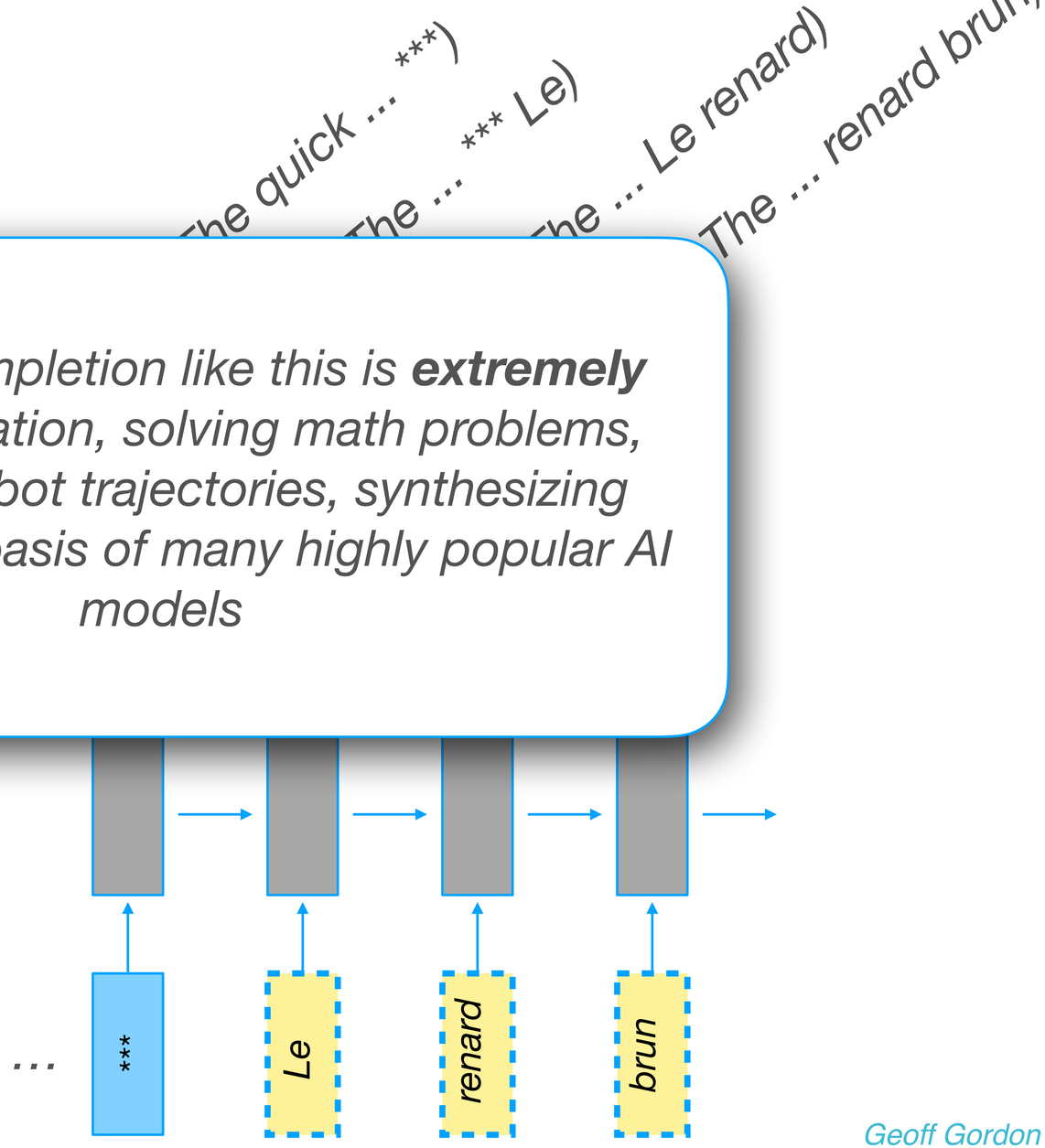
Geoff Gordon

$$P(\text{Le renard brun rapide} \mid \text{prompt}) = P(\text{Le} \mid \text{prompt}) \cdot P(\text{renard} \mid \text{prompt}, \text{Le}) \cdot \ldots$$

Generation process (sequence completion)

- Output token at each step is a sample from *conditional probability distrib* prece
- Like a with 5
  - wei out
  - roll, that lands face up
- Copy this token to input (condition on it) to generate next

*Sequence completion like this is **extremely** flexible: translation, solving math problems, generating robot trajectories, synthesizing images — the basis of many highly popular AI models*



Sampling from an RNN-LM

Training data (Shakespeare's As You Like It)

VIOLA: Why, Salisbury must find his flesh and thought
That which I am not aps, not a man and in fire,
To show the reining of the raven and the wars
To grace my hand reproach within, and not a fair are hand,
That Caesar and my goodly father's world;
When I was heaven of presence and our fleets,
We spare with hours, but cut thy council I am great,
Murdered and by thy master's ready there
My power to give thee but so much as hell:
Some service in the noble bondman here,
Would show him to her wine.

KING LEAR: O, if you were a feeble sight, the courtesy of your law,
Your sight and several breath, will wear the gods
With his heads, and my hands are wonder'd at the deeds,
So drop upon your lordship's head, and your opinion
Shall be against your honour.

Sampling from an RNN-LM

Sample (RNN-LM)

CHARLES: Marry, do I, sir; and I came to acquaint you with a matter. I am given, sir, secretly to understand that your younger brother Orlando hath a disposition to come in disguised against me to try a fall. To-morrow, sir, I wrestle for my credit; and he that escapes me without some broken limb shall acquit him well. Your brother is but young and tender; and, for your love, I would be loath to foil him, as I must, for my own honour, if he come in: therefore, out of my love to you, I came hither to acquaint you withal, that either you might stay him from his intendment or brook such disgrace well as he shall run into, in that it is a thing of his own search and altogether against my will.

TOUCHSTONE: For my part, I had rather bear with you than bear you; yet I should bear no cross if I did bear you, for I think you have no money in your purse.

Example from http://karpathy.github.io/2015/05/21/rnn-effectiveness/

Sampling from an RNN-LM

Sample (RNN-LM)

CHARLES: Marry, do I, sir; and I came to acquaint you with a matter. I am given, sir, secretly to understand that your younger brother Orlando hath a disposition to come in disguised against me to try a fall. To-n… capes me without s… her is but young and… il him, as I must, for… y love to you, I car… night stay him from… shall run into, in th… gainst my

… actually, this is the training data, and the previous one was the sample

TOUCHSTONE: For my part, I had rather bear with you than bear you; yet I should bear no cross if I did bear you, for I think you have no money in your purse.

Example from http://karpathy.github.io/2015/05/21/rnn-effectiveness/

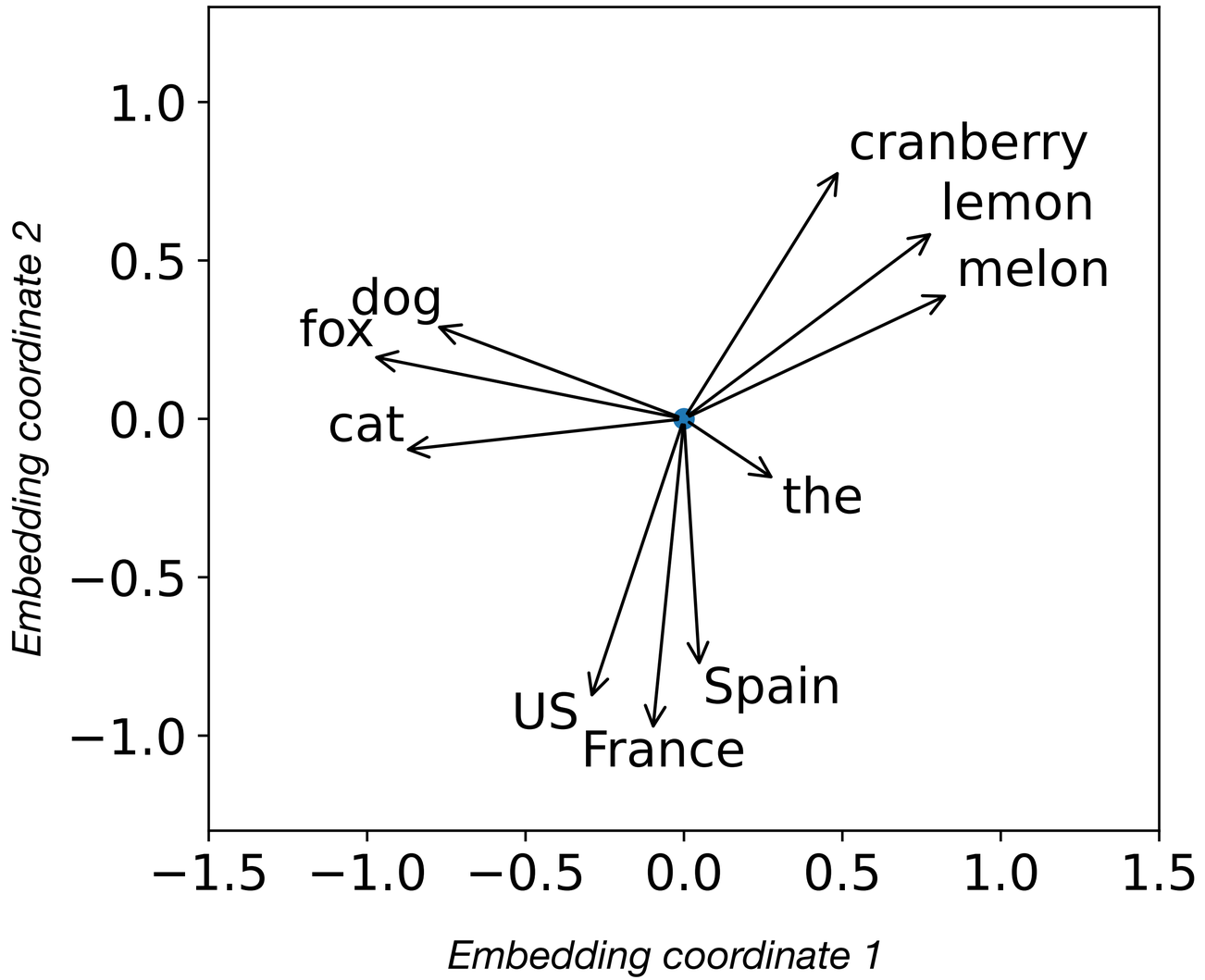
Learning goals: RNNs

- What different kinds of sequence prediction tasks might we encounter and how can we present them to an RNN?
- What does a basic RNN block look like, and how do we use it to make predictions?
- What is a vector-space embedding, and what is it used for?
- What is long-term memory in an RNN, how does it relate to gating, and what problem does it solve?
- What is backprop through time? How is it similar or different to backprop in other deep nets?
- How can we train an RNN language model? How do we use it to predict?

Vector-space embeddings

10-701 Introduction to Machine Learning
Geoff Gordon

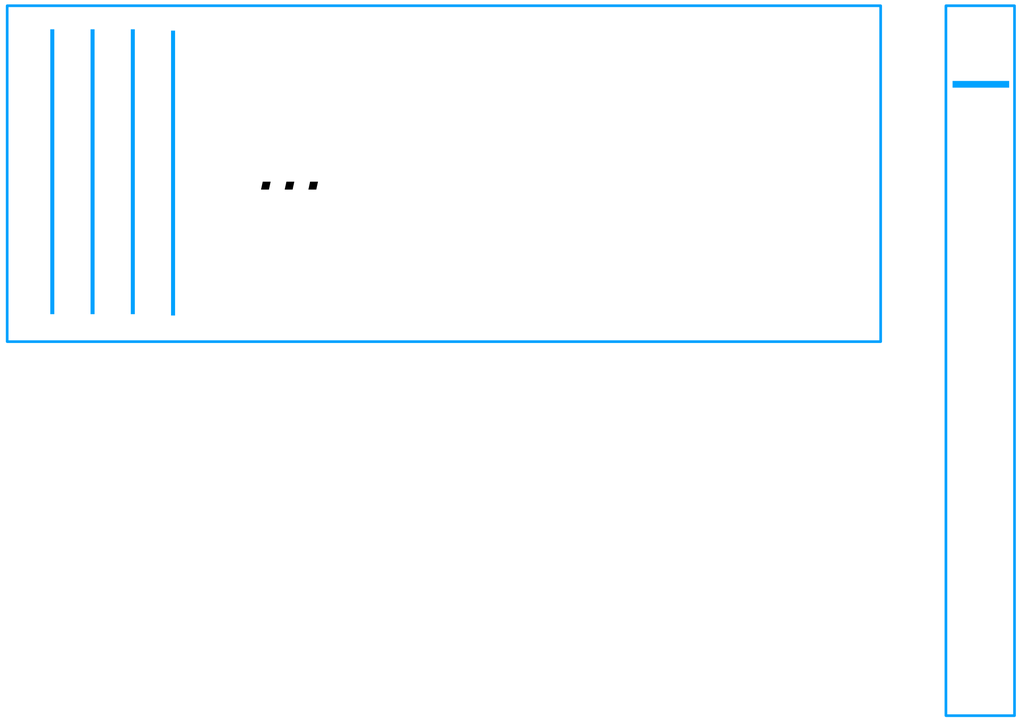
Vector-space embedding



Arrows = embeddings $\mathbf{v}_j$ *(here, in* $\mathbb{R}^2$*)*

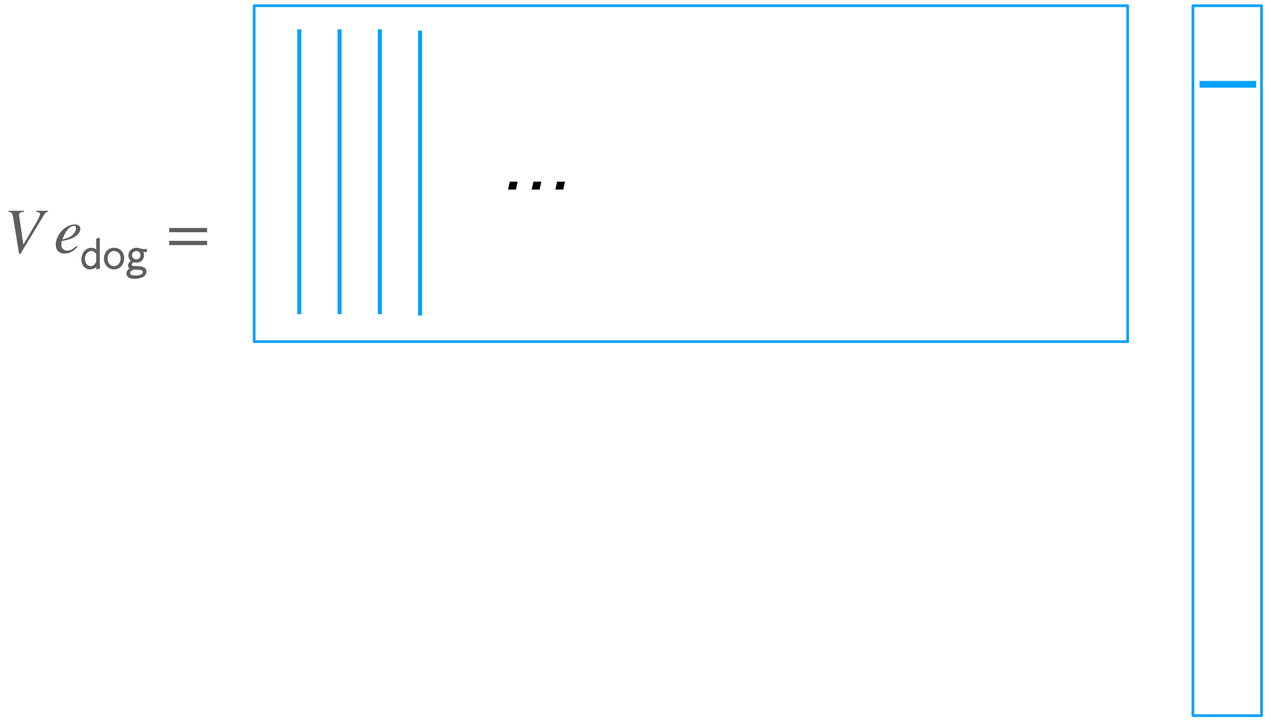
Where do embeddings come from?

$$V e_{\text{dog}} =$$



- If we present tokens as one-hot, first layer automatically learns a vector-space encoding $V = W_1$ as we train
- But we can often improve embeddings w/ a ***surrogate*** or ***proxy*** task
  - a task we don't directly care about, but which is related enough to help learning the task we do care about, and for which we can easily/cheaply get lots of training data

Where do embeddings come from?



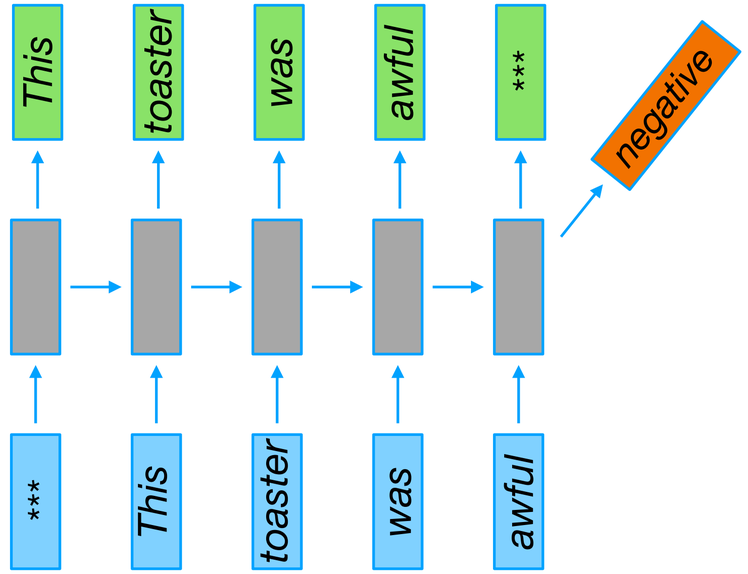
- If we present tokens as one-hot, first layer automatically learns a vector-space encoding $V = W_1$ as we train
- But we can often improve embeddings w/ a ***surrogate*** or ***proxy*** task — e.g., next-token prediction
  - a task we don't directly care about, but which is related enough to help learning the task we do care about, and for which we can easily/cheaply get lots of training data

Multi-task training

- Single model (e.g., RNN or MLP) implements both tasks
  - often via separate prediction heads
- Train to minimize:

  [loss for true task] + λ [loss for proxy task]
  - can even use several proxy tasks at once this way
  - when setting λ, remember losses are on different scales

Multi-task example

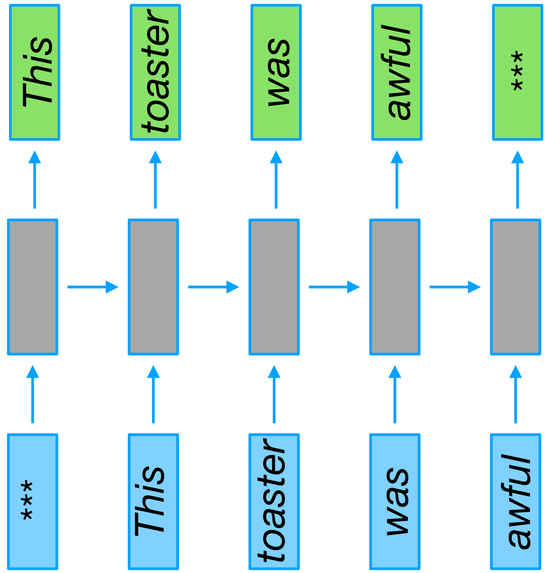


- For example, suppose we want to learn sentiment classification — is a product review positive or negative?
- Proxy task: next token prediction — easy to get huge amounts of data (scrape product reviews from web)
- Target task: sequence classifier — suppose we need expert labels → smaller dataset

Multi-task training

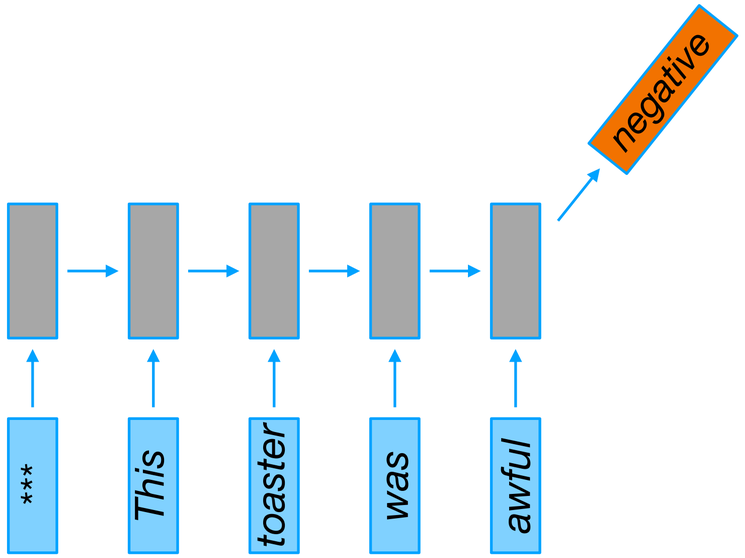
- Difficulty: proxy task training can be expensive (much larger training set, by design)
  - might want to amortize expense across several true tasks
  - or even process proxy task before we know true task(s)
  - difficult to do if we have to process everything at the same time
- Advantage: seems to regularize (limit complexity) in addition to letting us extract info from proxy task data

Pre-training



- Or, we could train first on the proxy task (pre-training)
- Then switch to the “true” task (fine-tuning)
  - rely on SGD’s gradual nature: we still remember pre-training as we fine-tune
  - as before, different prediction head for each task
- Can generalize to a sequence of training tasks/phases, with names like fine-tuning, mid-training, post-training

Pre-training

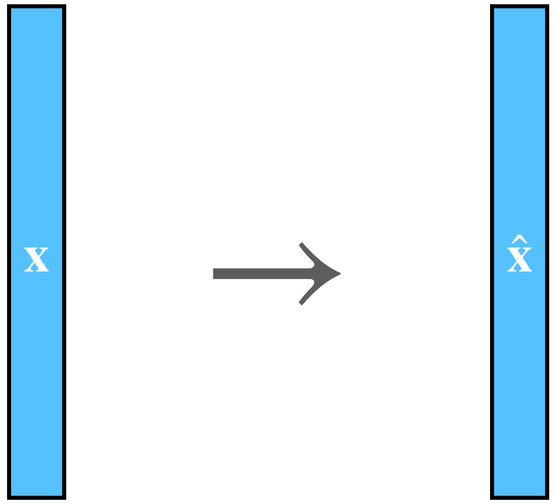


- Or, we could train first on the proxy task (pre-training)
- Then switch to the “true” task (fine-tuning)
  - rely on SGD’s gradual nature: we still remember pre-training as we fine-tune
  - as before, different prediction head for each task
- Can generalize to a sequence of training tasks/phases, with names like fine-tuning, mid-training, post-training

Pre-training

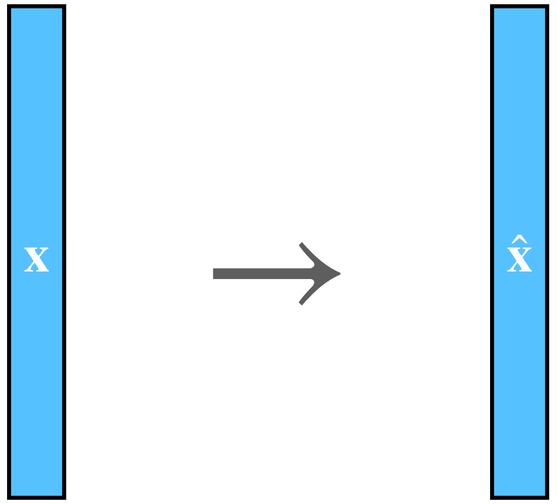
- Difficulty: forgetting
  - minimize by using a smaller learning rate (often at least an order of magnitude smaller) and early stopping
  - or by learning an explicit offset and regularizing it (weight decay, limited rank weight matrices (LoRA))
- Advantage: pre-train once, fine-tune as often as we like
- Most current large-scale approaches prefer pre-training to multi-task learning, but some use both

Proxy task: autoencoder



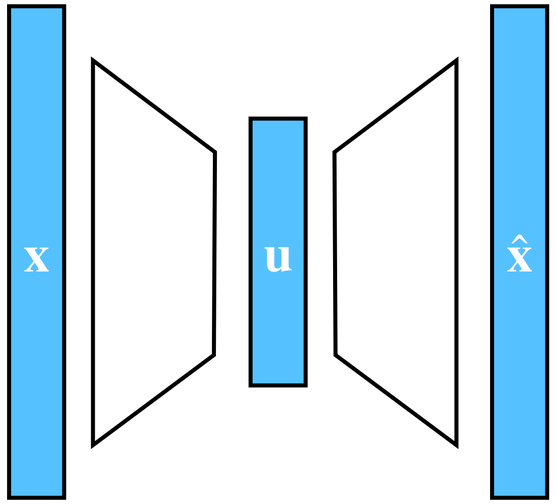
- Predict features or tokens $\mathbf{x}^{(i)}$ from input $\mathbf{x}^{(i)}$

Proxy task: autoencoder



- Predict features or tokens $\mathbf{x}^{(i)}$ from input $\mathbf{x}^{(i)}$
  - seems kind of easy

Proxy task: autoencoder



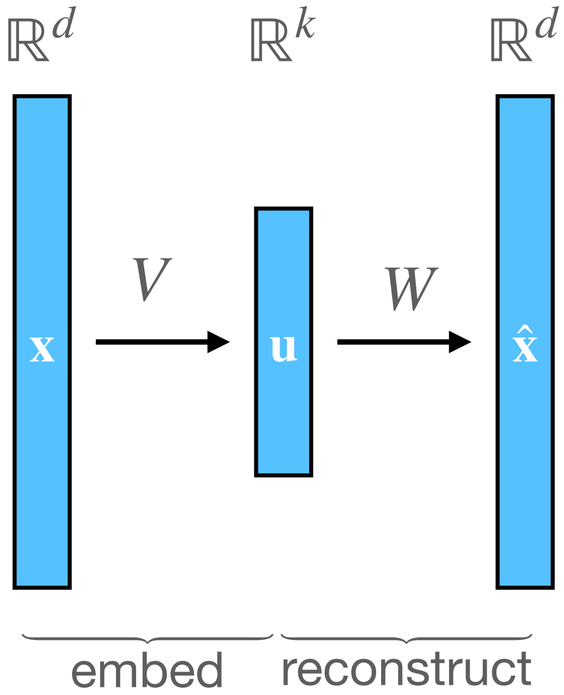
Why the name? "Code" = hidden activations, so $\mathbf{x}$ encodes (and then decodes) itself

- Predict features or tokens $\mathbf{x}^{(i)}$ from input $\mathbf{x}^{(i)}$
  - seems kind of easy
- The catch: something about the model (the ***bottleneck***) prevents us from just copying input to output
  - e.g., hidden layer $\mathbf{u}^{(i)}$ w/ too few dimensions
  - e.g., regularizer that disfavors straight copying
  - e.g., connection pattern that prevents copying — saw this already in masked token prediction

Why auto-encoding?

- Autoencoding is a good proxy task because:
  - it's related to many other tasks: the hidden vector $\mathbf{u}$ has to be a decent summary of the input
  - it's really easy to get training data: e.g., for language, any natural string of tokens is fair game (a novel, a blog post, a product review, a newspaper article)
- Hyperparameter: granularity
  - could pass in an entire newspaper article, or split it into paragraphs, or take smaller fixed-length windows of tokens

Linear autoencoder



$$\hat{\mathbf{x}} = W\mathbf{u} = WV\mathbf{x}$$

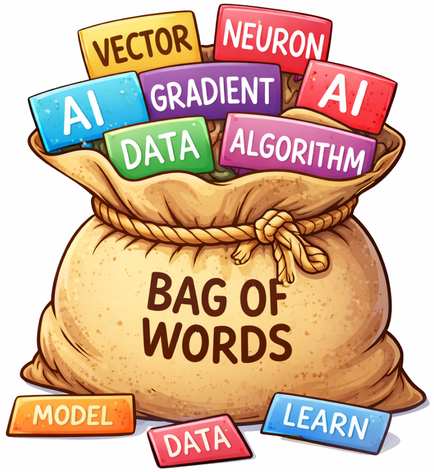
$$V \in \mathbb{R}^{k \times d}, W \in \mathbb{R}^{d \times k}$$

$$k \ll d$$

- Simplest autoencoder: one hidden layer, no nonlinearities
  - min sum squared error: $\min_{V,W} \sum_{i=1}^{N} \|WV\mathbf{x}^{(i)} - \mathbf{x}^{(i)}\|^2$
  - called *reconstruction error*
  - can solve e.g. w/ SGD or linear algebra

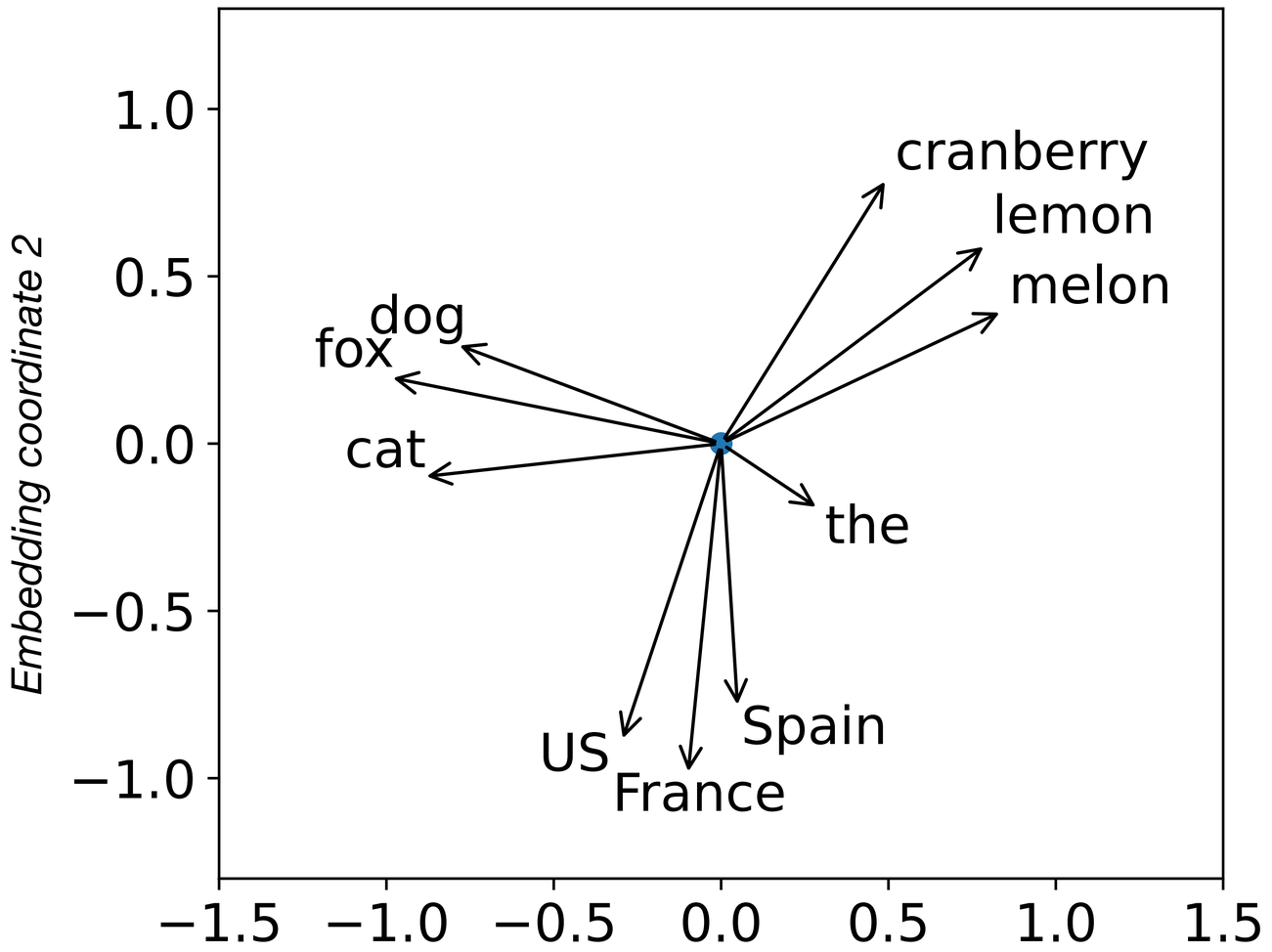
*Linear autoencoders are strongly related to the **principal components analysis** model (covered later)*

Bag of words



- One way to get vector-space embeddings: fit a linear autoencoder to a dataset of strings of tokens
- How can we pass a token string to linear autoencoder?
  - simplest way: sum up all the tokens, “quick brown fox” $\rightarrow \mathbf{e}_{\text{quick}} + \mathbf{e}_{\text{brown}} + \mathbf{e}_{\text{fox}} \rightarrow \mathbf{u} = \mathbf{v}_{\text{quick}} + \mathbf{v}_{\text{brown}} + \mathbf{v}_{\text{fox}}$
  - permutation-invariant (order doesn’t matter)
  - i.e., a ***multiset*** or ***bag*** of words
- BoW assumption is only to make model simpler: in reality “man bites dog” is different from “dog bites man”

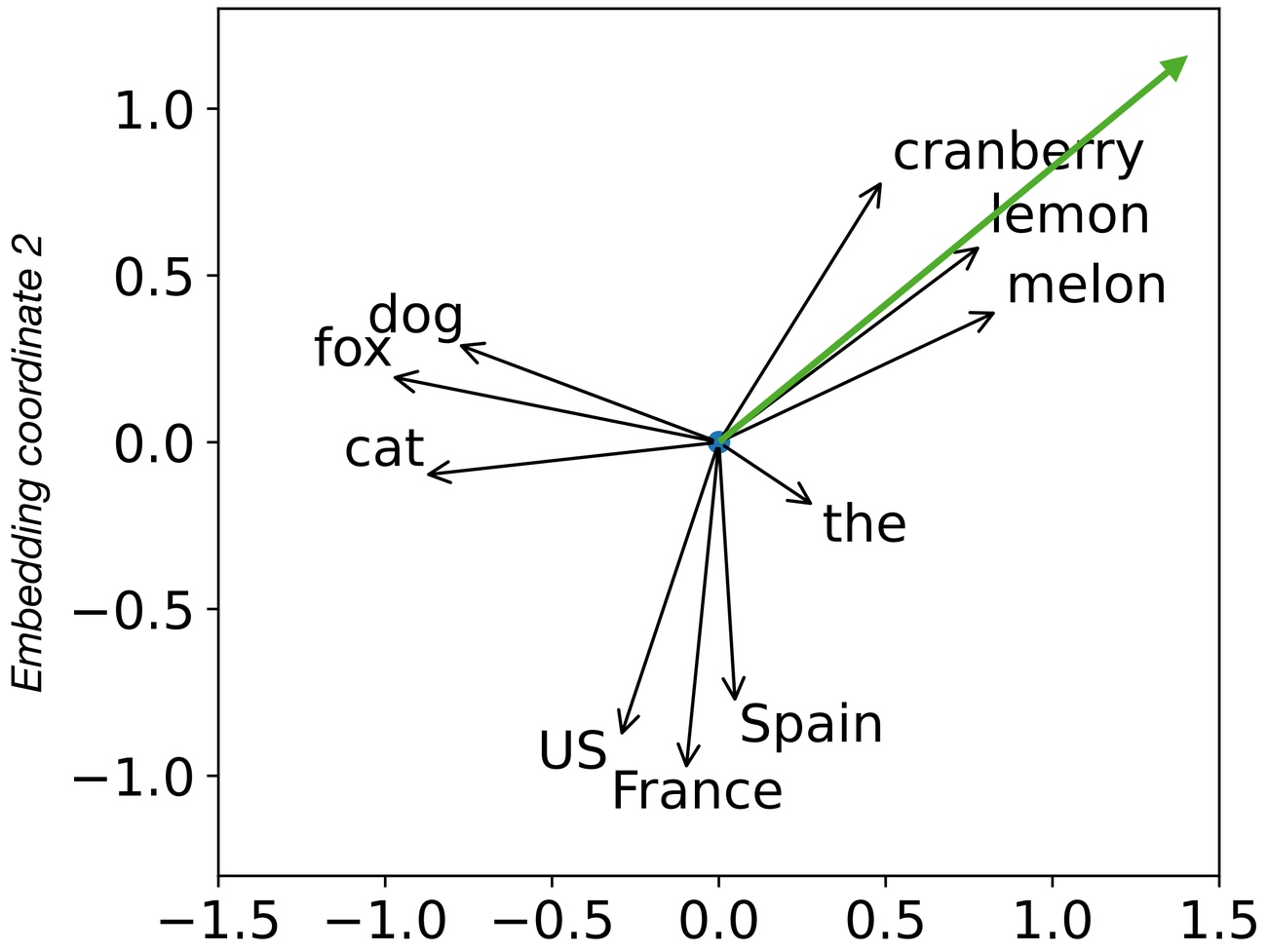
BoW example



Arrows = embeddings $\mathbf{v}_j$ *(here, in* $\mathbb{R}^2$*)*

- Embed “cranberry melon”

BoW example

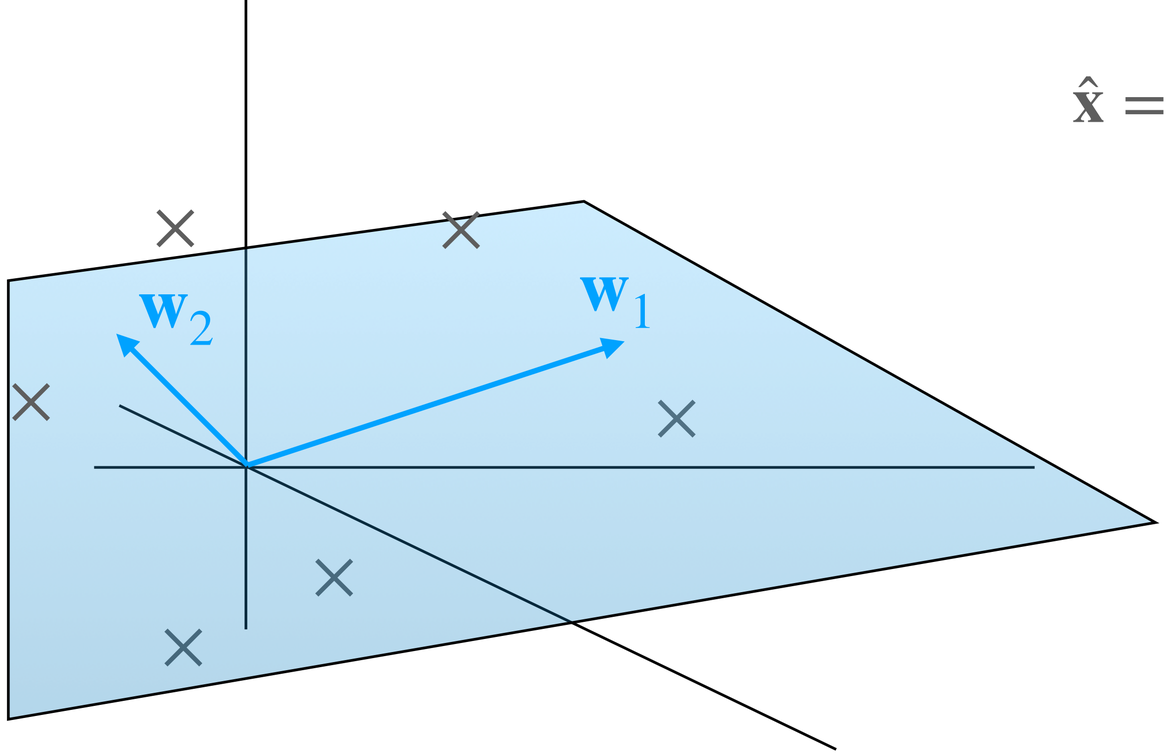


Arrows = embeddings $\mathbf{v}_j$ *(here, in* $\mathbb{R}^2$*)*

- Embed “cranberry melon”

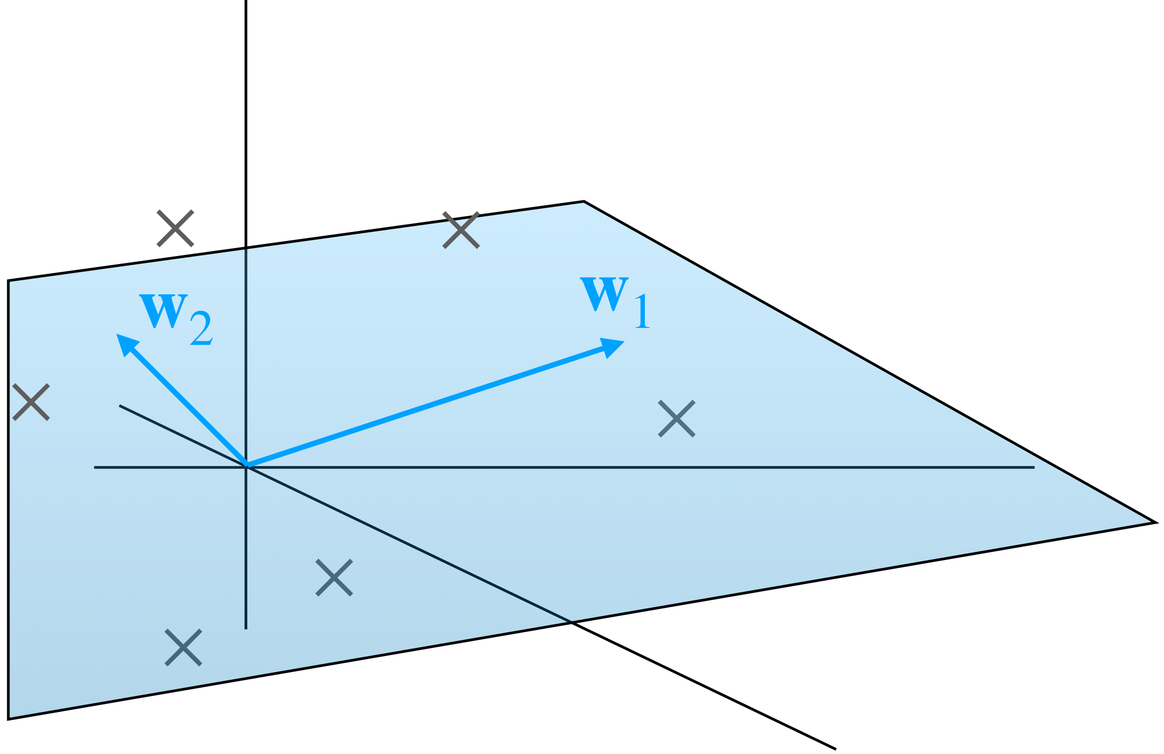
Interpreting autoencoder weights and activations

$$\hat{\mathbf{x}} = W\mathbf{u} = WV\mathbf{x}$$



- Columns $\mathbf{w}_j$ of W: a basis for reconstructed examples $\hat{\mathbf{x}}^{(i)}$ (predicted bags of tokens)
- Hidden vectors $\mathbf{u}^{(i)}$: coefficients for the basis vectors in the representation of $\hat{\mathbf{x}}^{(i)}$ — this is our embedding
- Columns of V: what will $\mathbf{u}^{(i)}$ be if input $\mathbf{x}^{(i)}$ is a unit vector — i.e., column for $\mathbf{e}_{\text{dog}}$ is embedding of dog

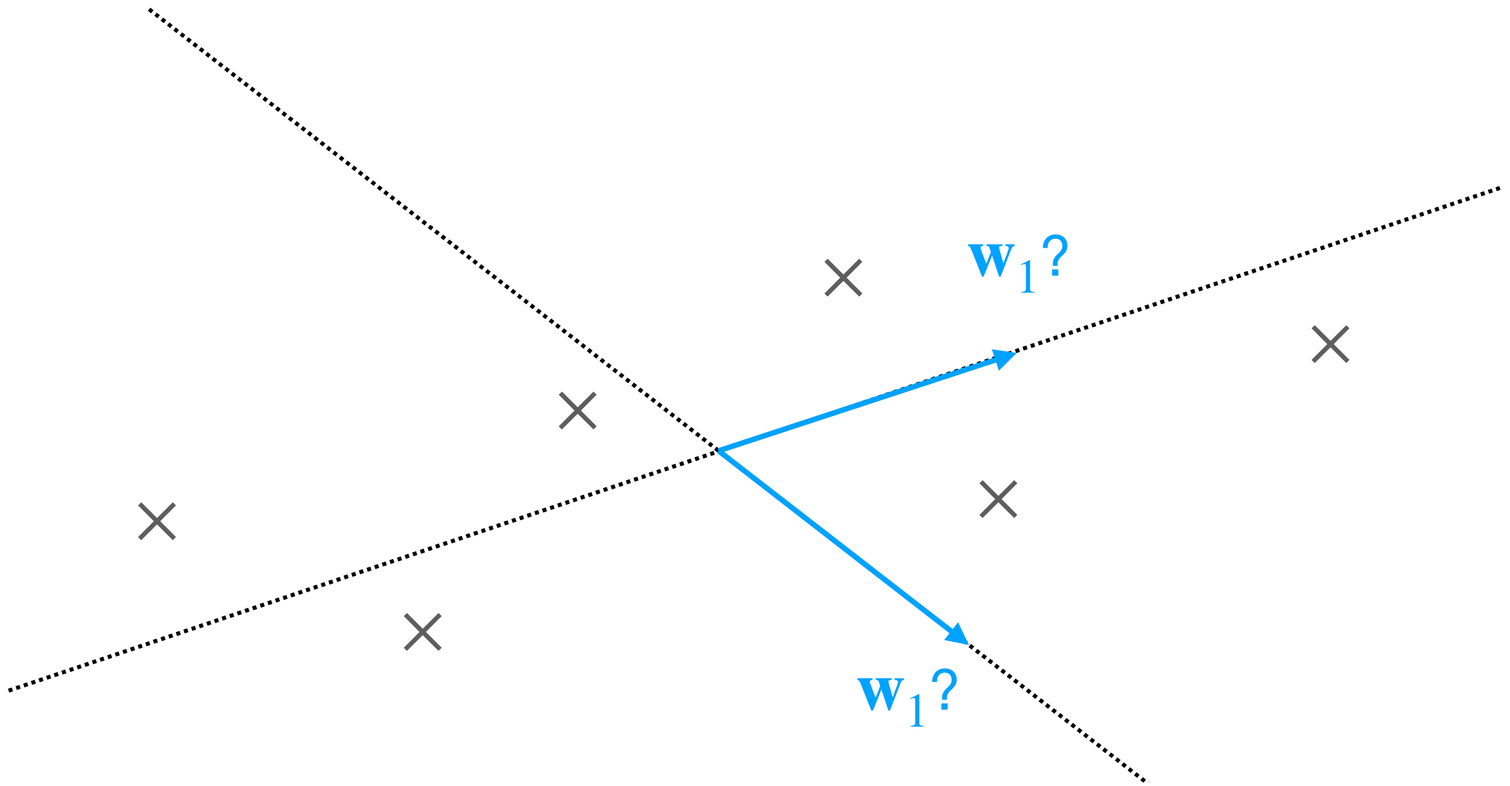
Interpreting autoencoder weights and activations



- Given optimal W, the matching V is unique (and vice versa)
  - can find it by SGD or linear algebra: it's $V = W^\dagger$
  - best V finds closest $\hat{\mathbf{x}}$ to a given $\mathbf{x}$ within the subspace we can represent — i.e., it projects onto basis $\{\mathbf{w}_j\}$

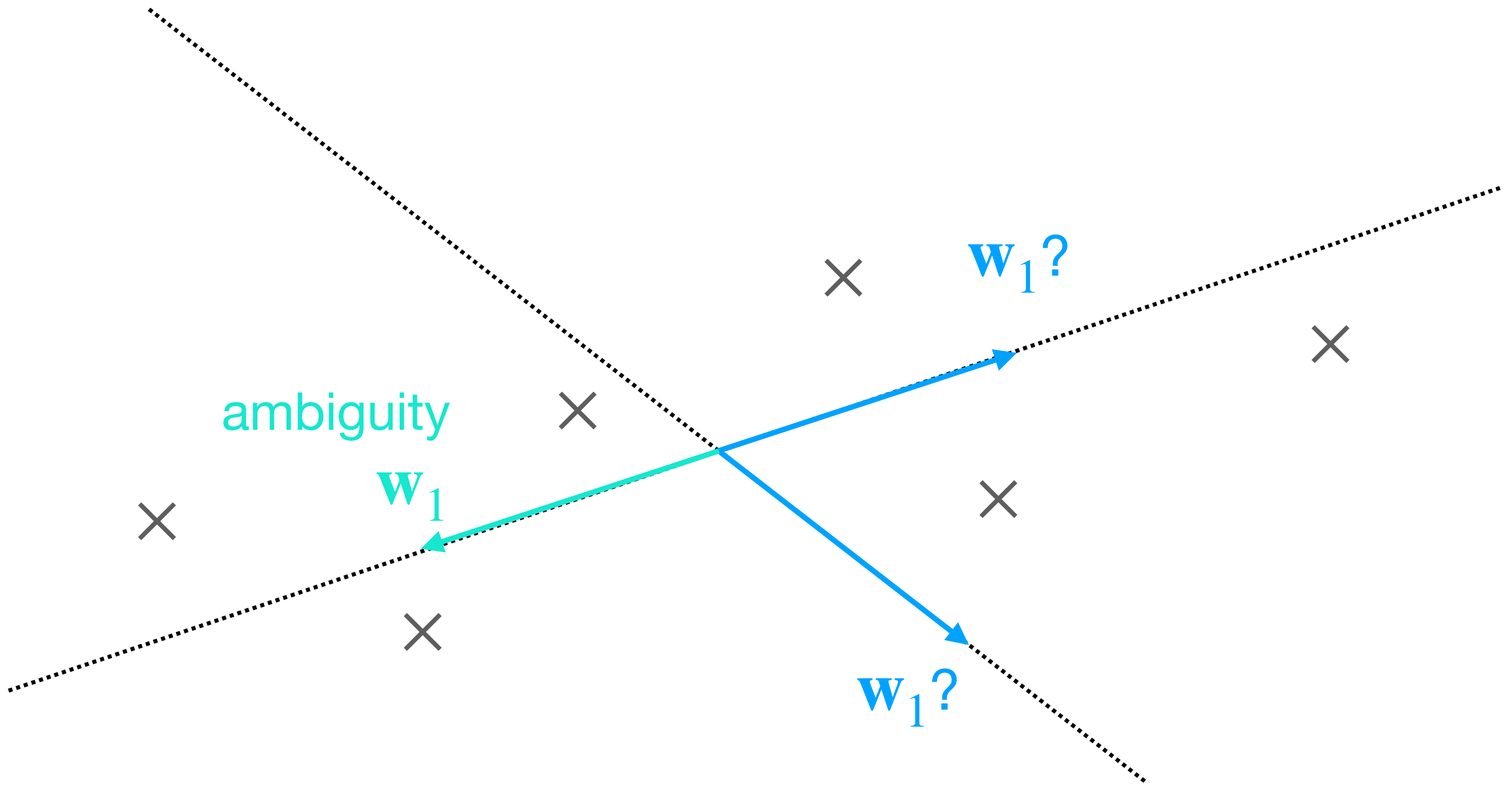
1-d autoencoder

First col of $W = \mathbf{w}_1$



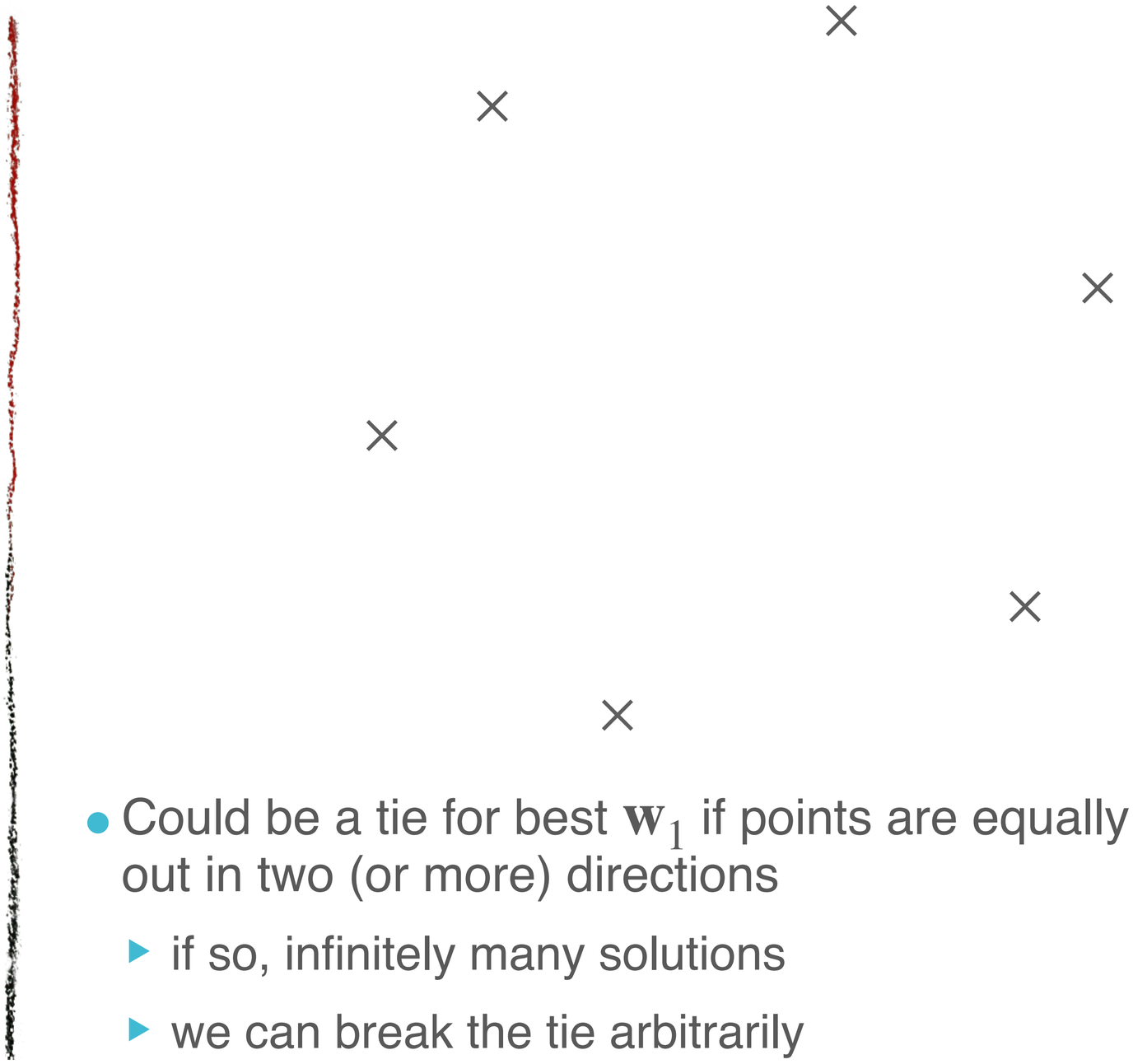
- Visualize solution to 1-d autoencoder
  - min sum-squared error: pick $\mathbf{w}_1$
  - intuitively: if datapoints are spread out, $\mathbf{w}_1$ should point along the long direction

1-d autoencoder



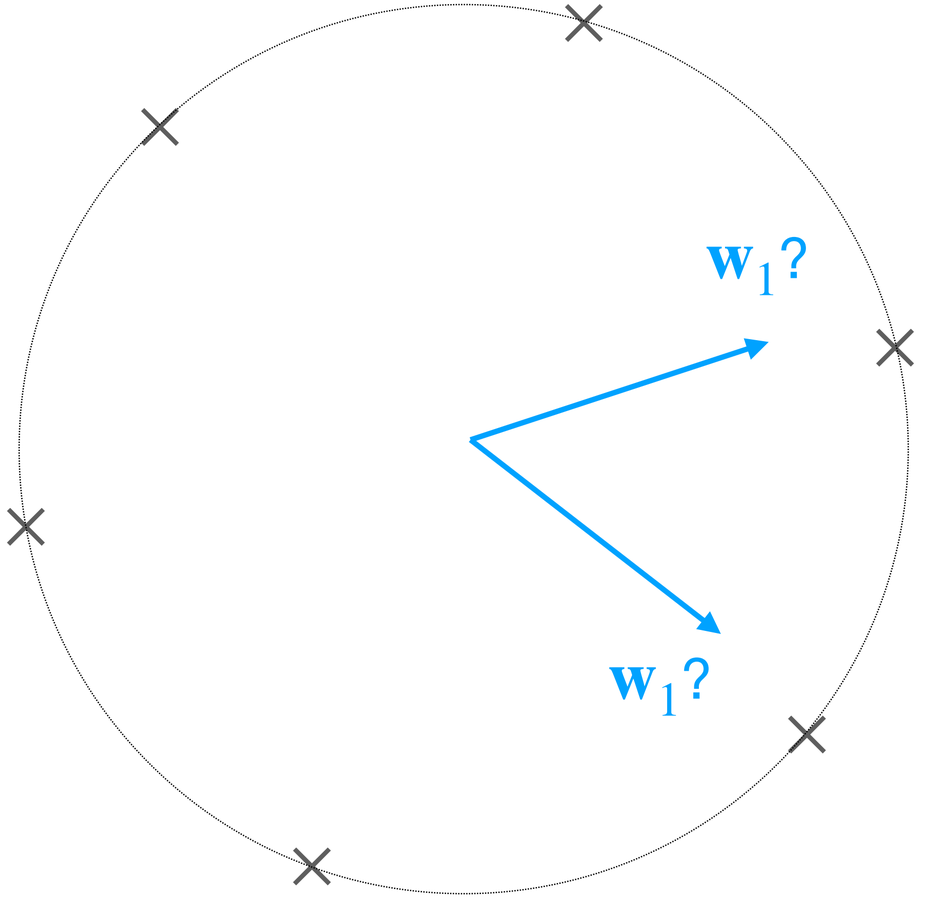
- Visualize solution to 1-d autoencoder
  - min sum-squared error: pick $\mathbf{w}_1$
  - intuitively: if datapoints are spread out, $\mathbf{w}_1$ should point along the long direction

Ambiguity



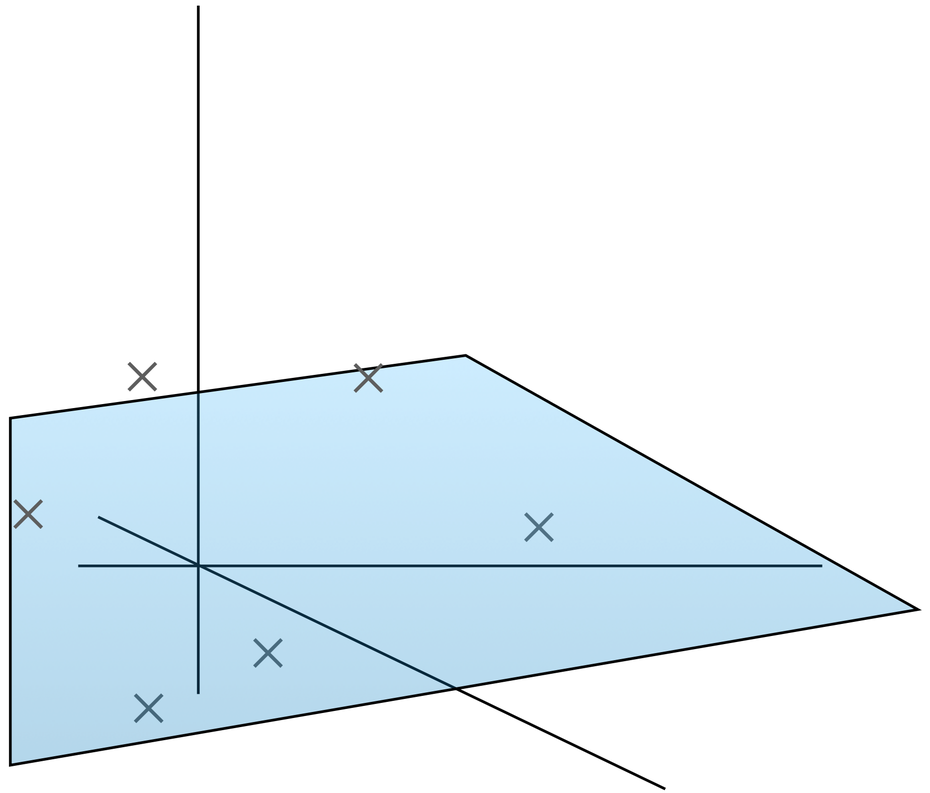
- Could be a tie for best $\mathbf{w}_1$ if points are equally spread out in two (or more) directions
  - if so, infinitely many solutions
  - we can break the tie arbitrarily

Ambiguity



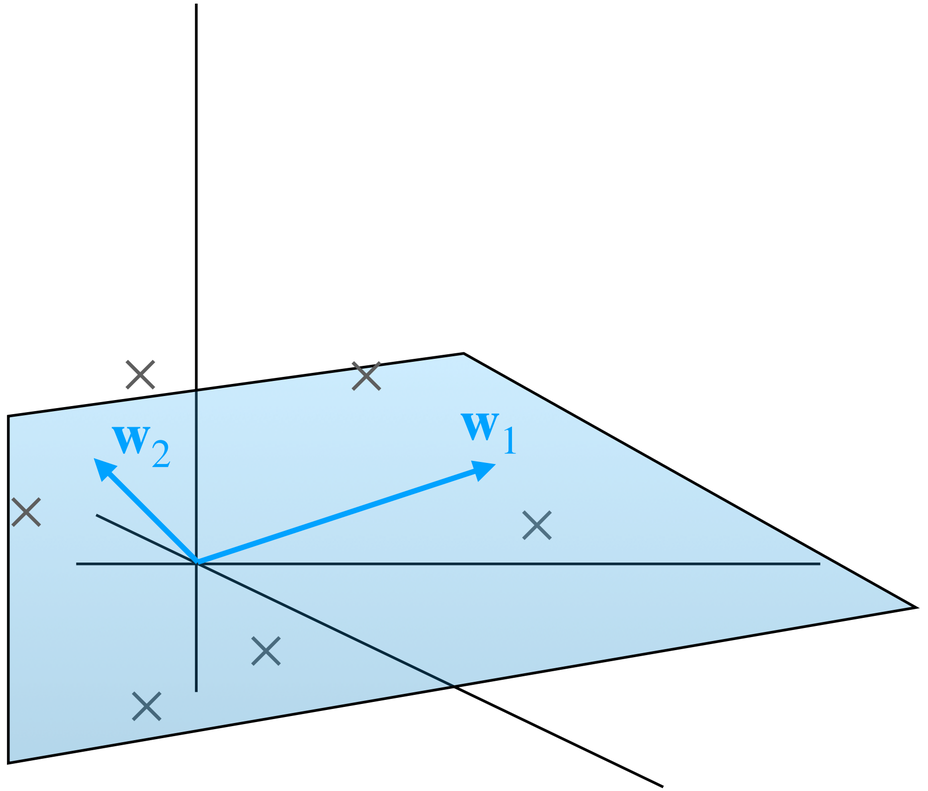
- Could be a tie for best $\mathbf{w}_1$ if points are equally spread out in two (or more) directions
  - if so, infinitely many solutions
  - we can break the tie arbitrarily

Best plane



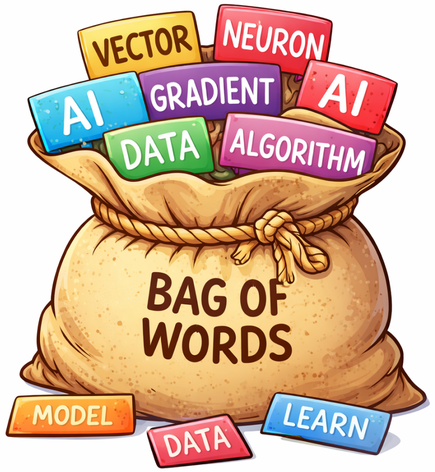
- What about $k = 2$?
  - project onto best 2D plane (min squared error)

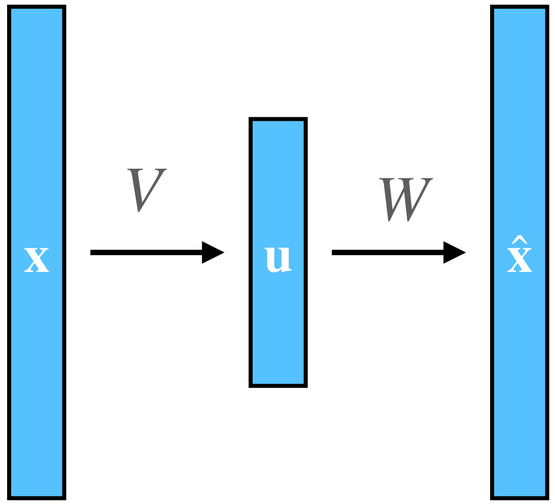
Ambiguity: basis



- Can choose any basis for the plane — reconstruction (and reconstruction error) will be the same
- Can (mostly) resolve this ambiguity by requiring $V = W^\top$
  - embeddings = columns of V = rows of W
  - basis = rows of V = columns of W

LSA

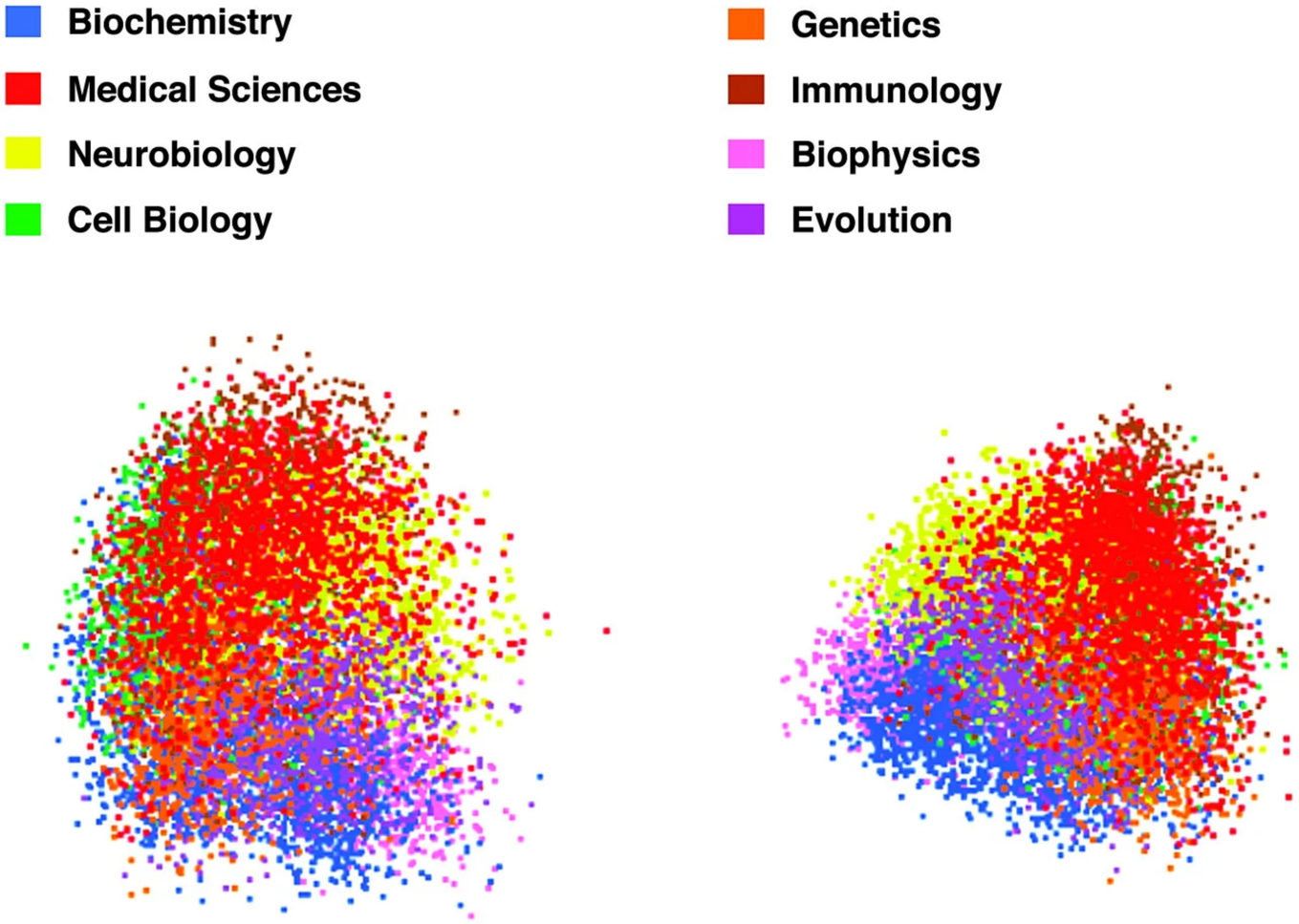




- OK, we got vector-space embeddings by fitting a linear autoencoder to a dataset of bags of tokens
  - maybe each bag is a document, or maybe it's a shorter window of consecutive tokens
- This model is called ***latent semantic analysis***
  - details: stopwords, IDF weighting, no mean subtraction

LSA example

Landauer et al., PNAS 2004 https://www.pnas.org/doi/10.1073/pnas.0400341101



- Ex: LSA of articles from PNAS shows that article embeddings reveal broad topic

Word2vec

- Early successful *nonlinear* embedding learner
- Train embedding matrix V by masked token prediction
  - reconstruction weights are tied, just as in LSA
- Two layer network:
  - first layer: embed and sum context tokens (BoW)

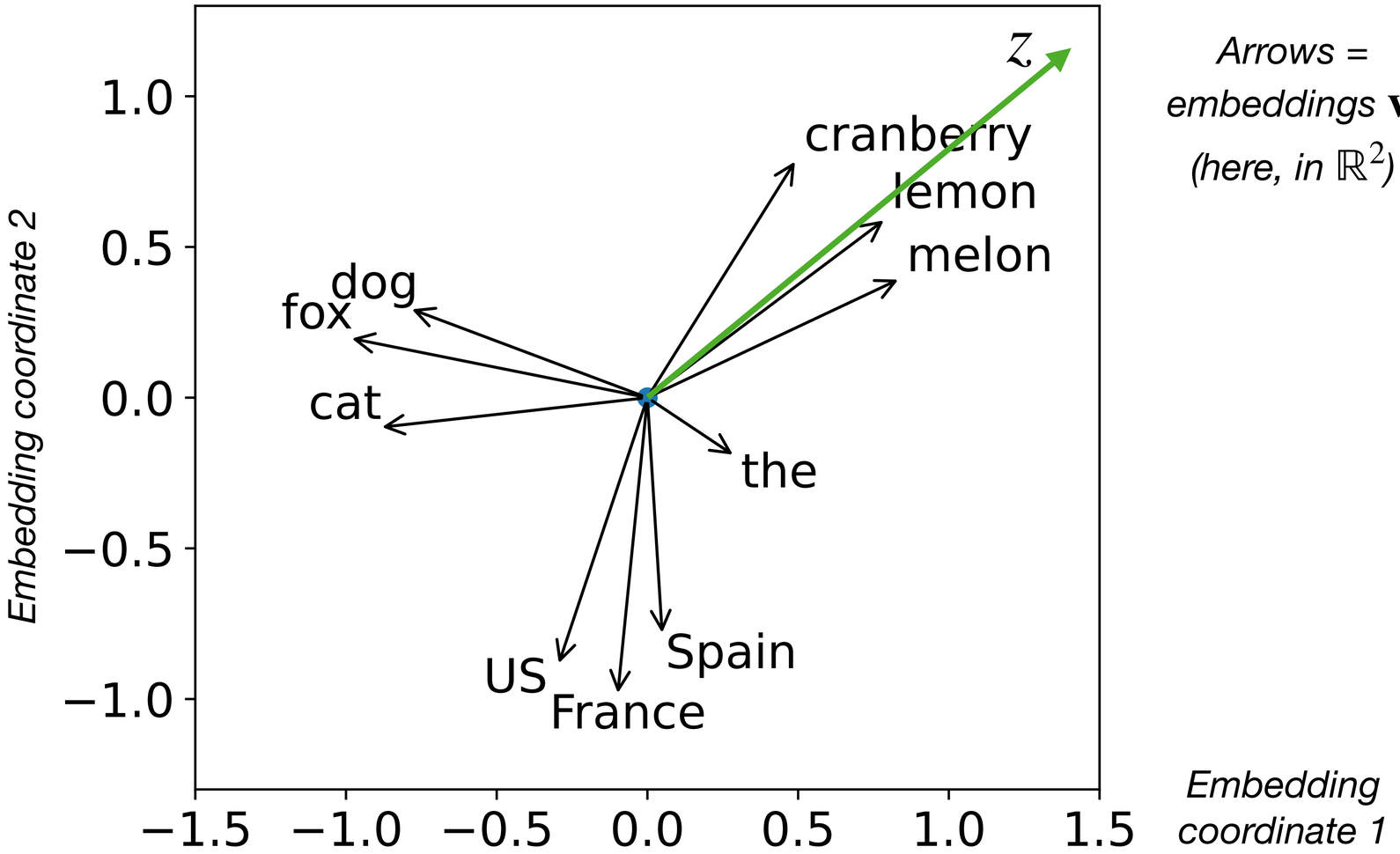
$$\mathbf{z} = \sum_{j=i-k,\, j\neq 0}^{i+k} V\mathbf{x}_j$$

  - second layer: softmax over vocabulary

$$P(\mathbf{y} \mid \text{input}) = \frac{\exp(\mathbf{z} \cdot V\mathbf{y})}{\sum_{\mathbf{y}'} \exp(\mathbf{z} \cdot V\mathbf{y}')}$$

  - note: approximations to make it fast

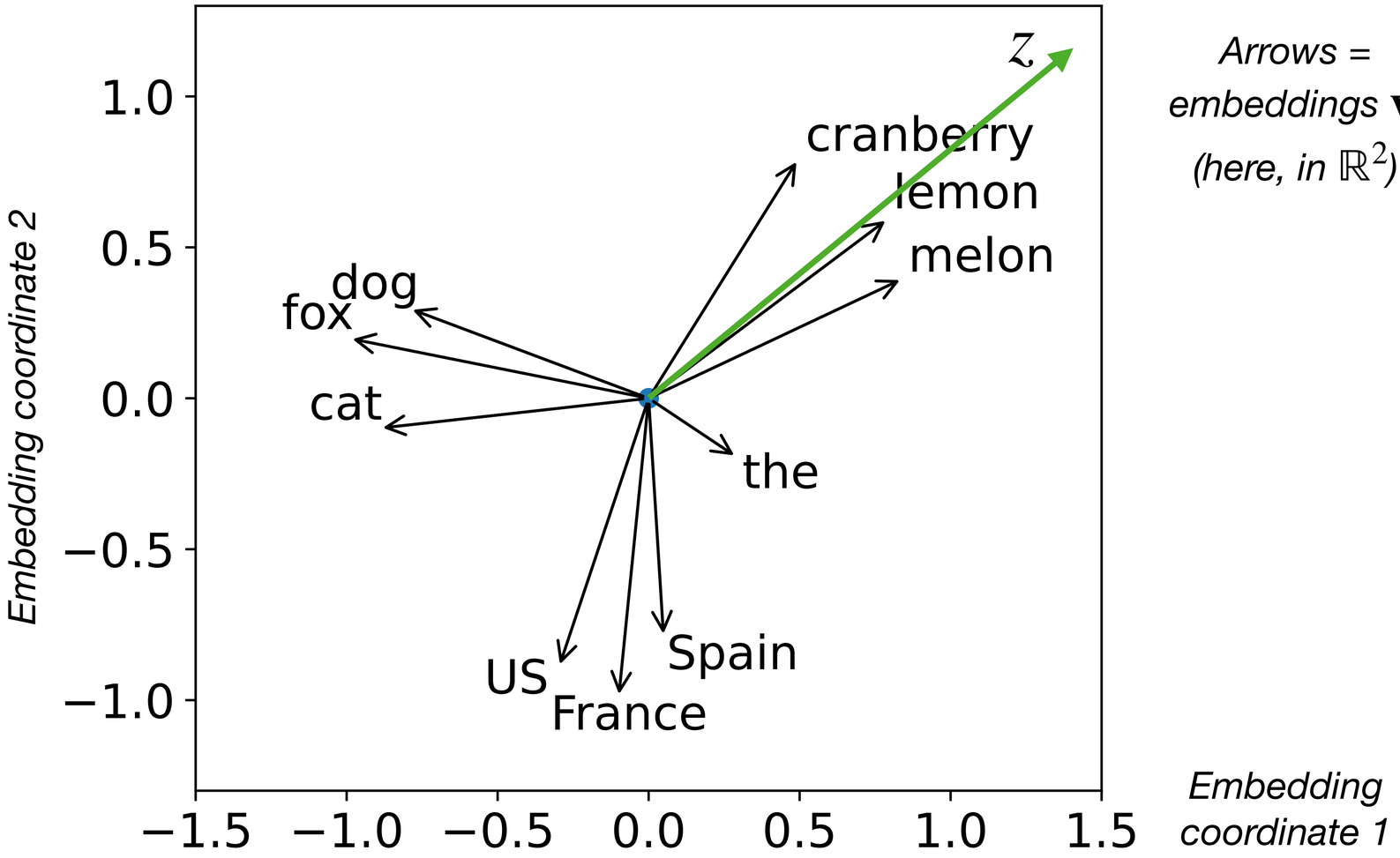
First layer



prediction task: … cranberry [???] melon …

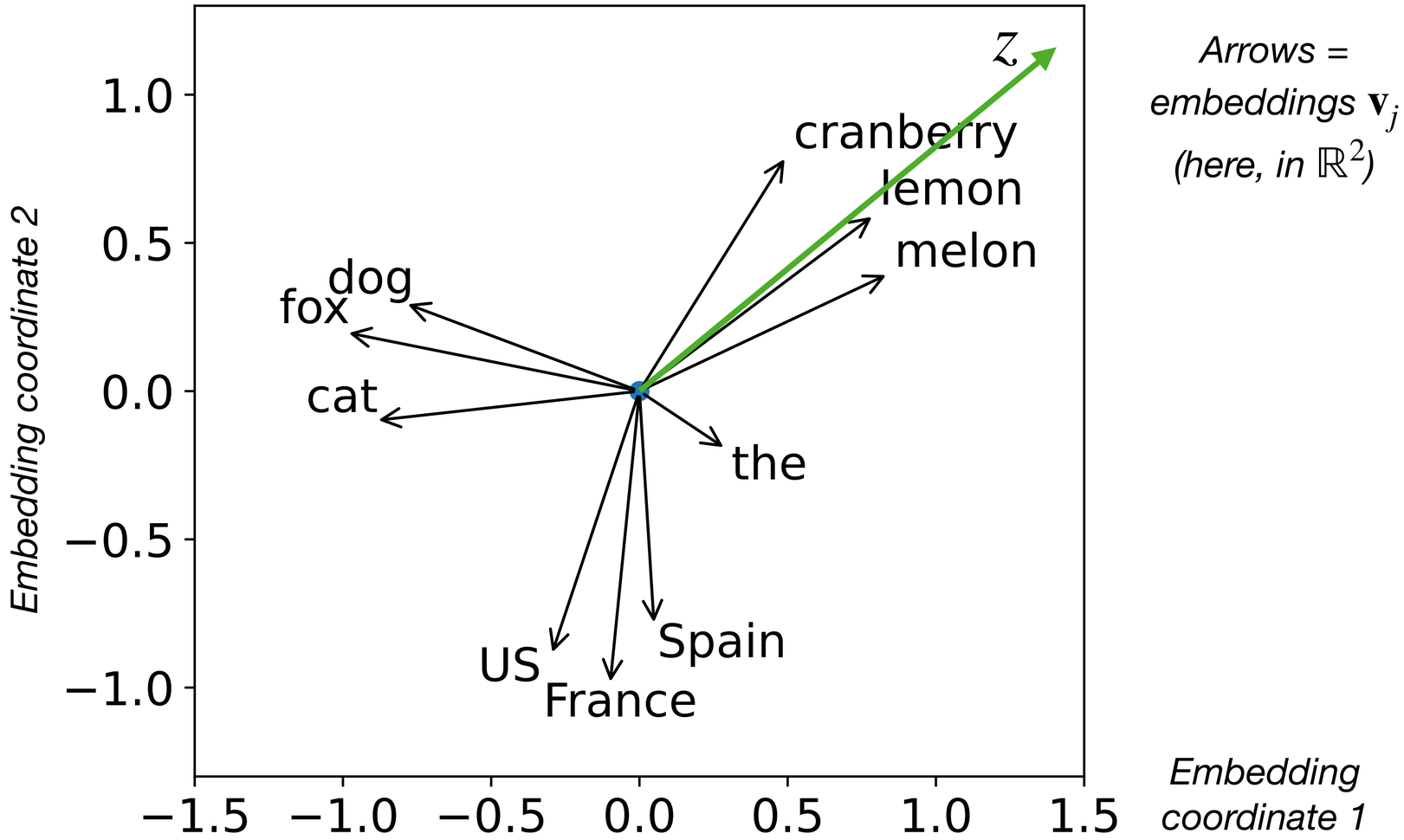
- $\mathbf{z}$ = sum of context tokens (i.e., context as a BoW)

Second layer



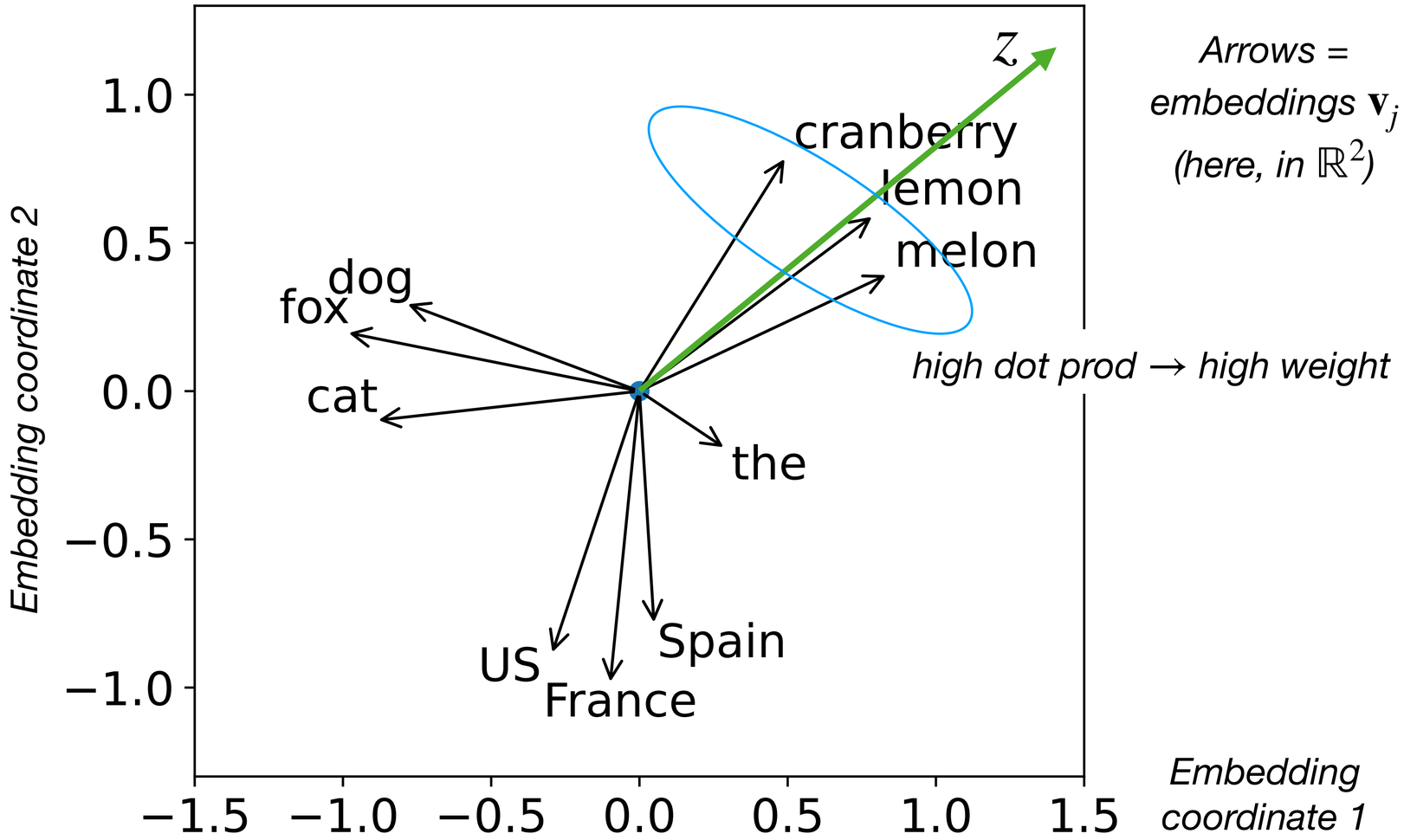
- Input to second layer = a vector $\mathbf{z}$, summary of context
- Goal of second layer: search through a set of vectors (vocabulary embeddings) to find similar ones to context

Second layer



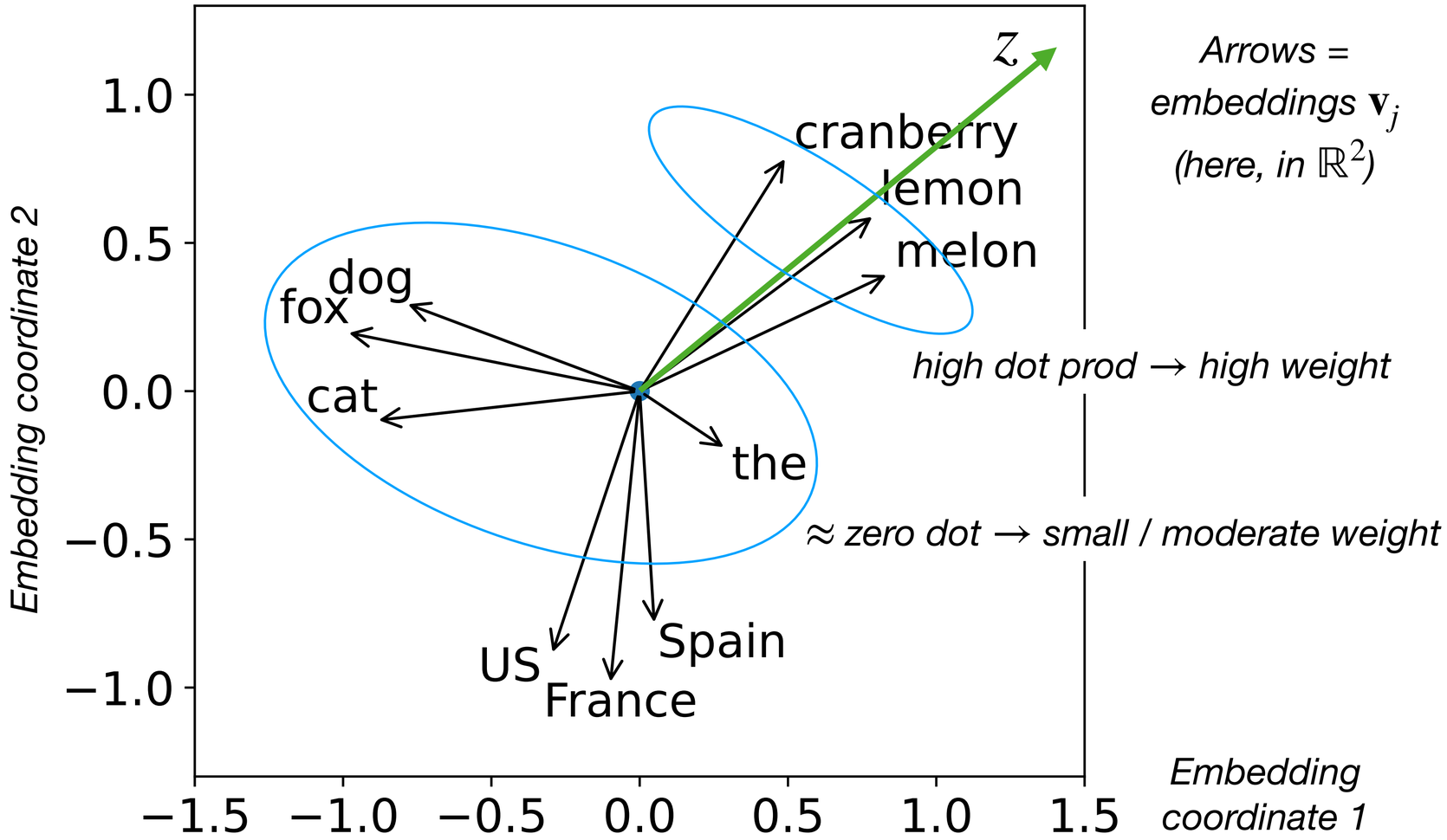
- Dot product $\mathbf{v}_j \cdot \mathbf{z}$ is highest for token embeddings that are both *high norm* and *similar direction* to $\mathbf{z}$
- The $\exp$ in softmax further emphasizes highest values

Second layer



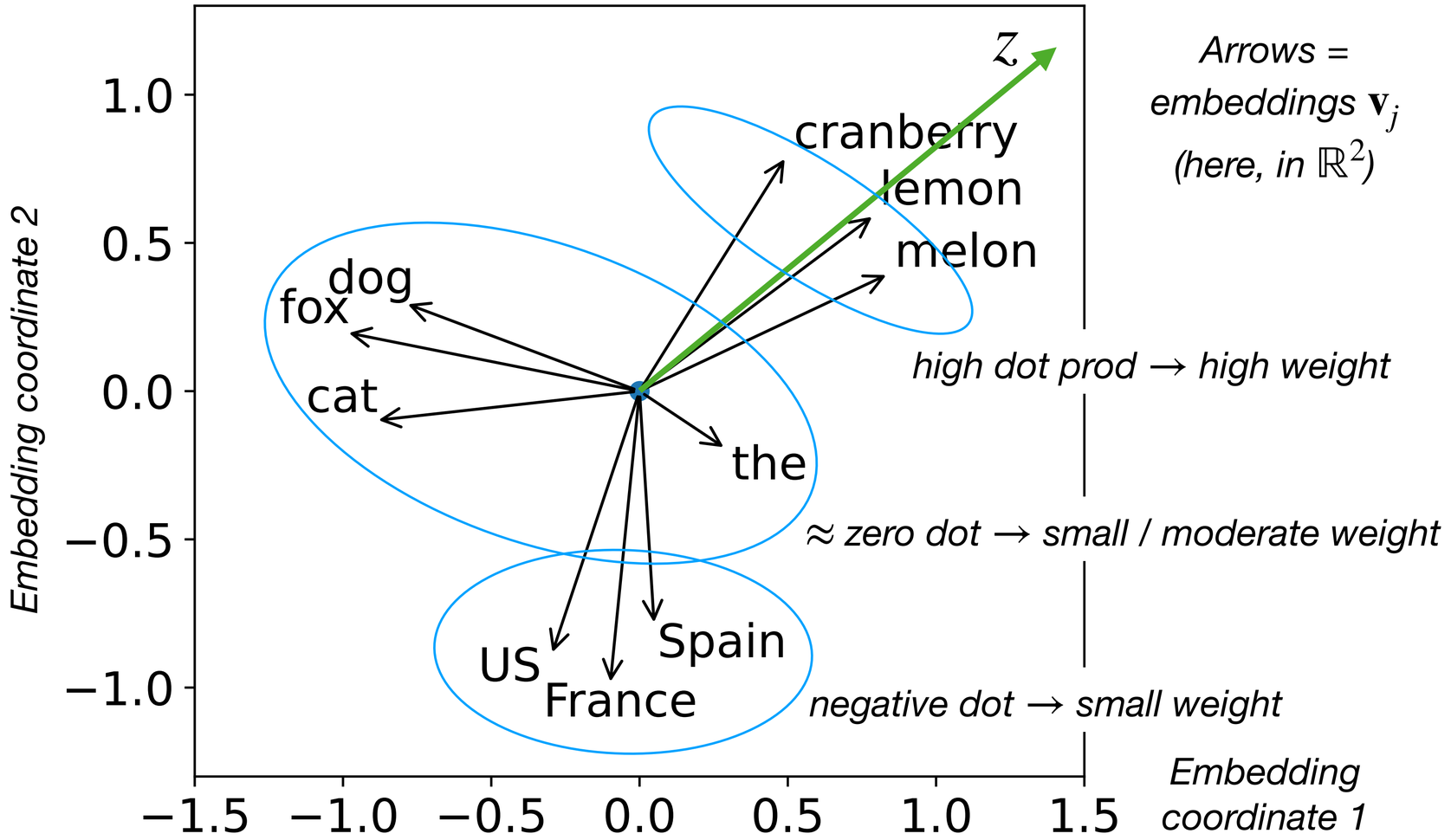
- Dot product $\mathbf{v}_j \cdot \mathbf{z}$ is highest for token embeddings that are both *high norm* and *similar direction* to $\mathbf{z}$
- The $\exp$ in softmax further emphasizes highest values

Second layer



- Dot product $\mathbf{v}_j \cdot \mathbf{z}$ is highest for token embeddings that are both *high norm* and *similar direction* to $\mathbf{z}$
- The $\exp$ in softmax further emphasizes highest values

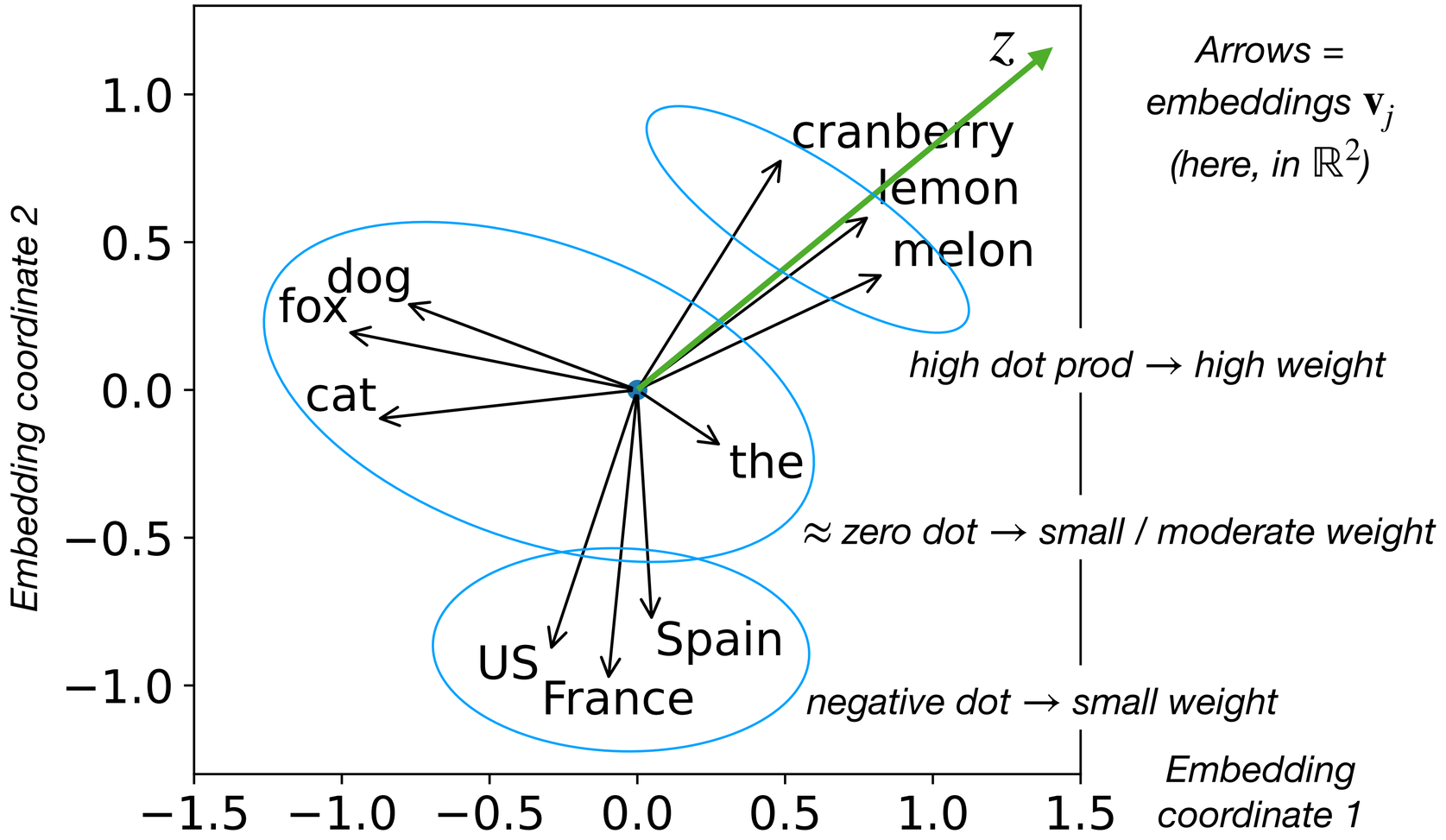
Second layer



- Dot product $\mathbf{v}_j \cdot \mathbf{z}$ is highest for token embeddings that are both *high norm* and *similar direction* to $\mathbf{z}$
- The $\exp$ in softmax further emphasizes highest values

Second layer

*This use of softmax (to find vocabulary words that have high dot product w/ context) is strongly related to **attention** (coming up soon)*

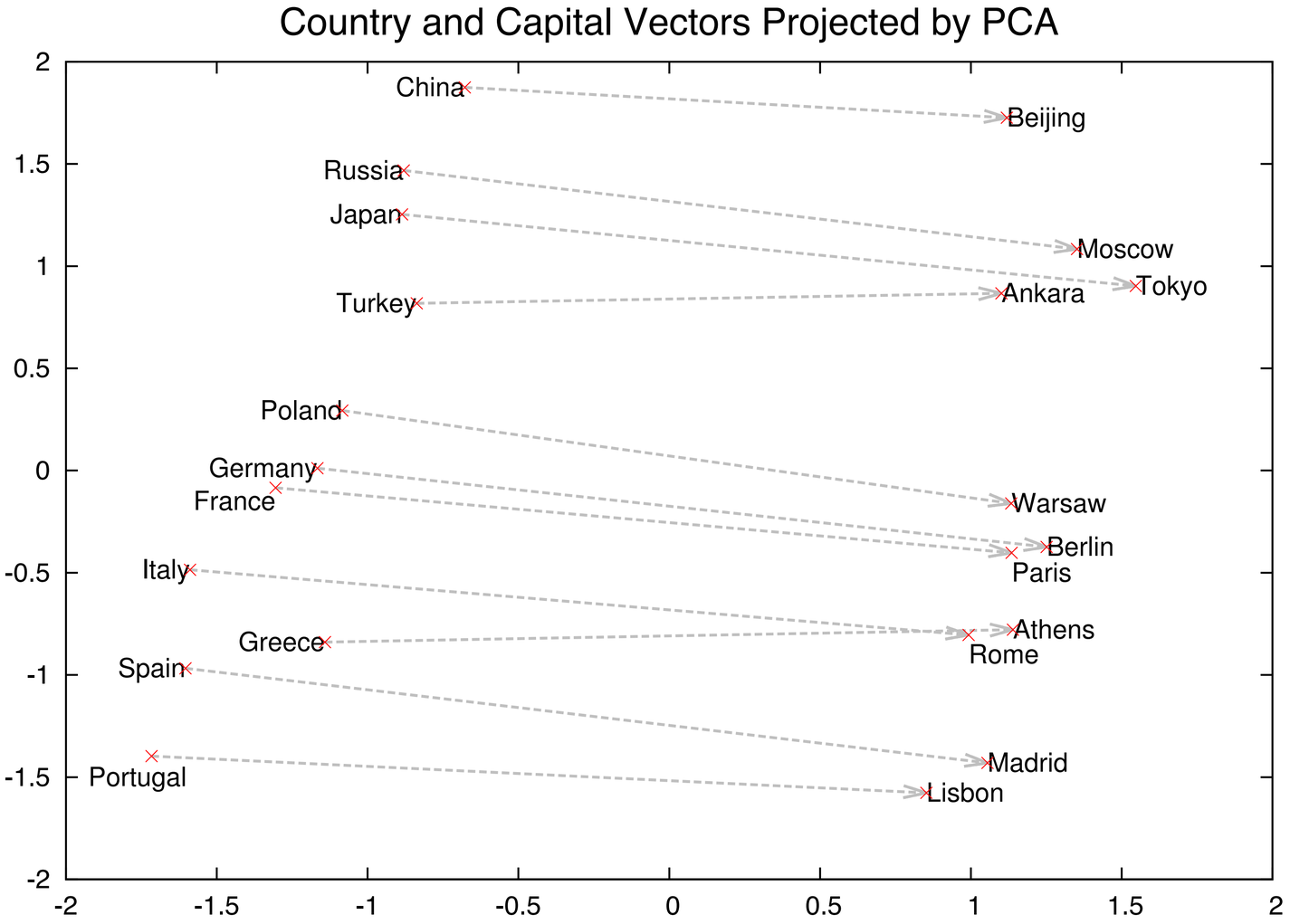


- Dot product $\mathbf{v}_j \cdot \mathbf{z}$ is highest for token embeddings that are both *high norm* and *similar direction* to $\mathbf{z}$
- The $\exp$ in softmax further emphasizes highest values

SGD step

- Each optimization step will try to
  - shift embedding of (words in) the query to better retrieve correct masked word (put more attention weight on it)
  - shift the correct masked word's embedding to better match query (have higher dot product with it)
  - shift all other words' embeddings to have lower dot products with the query

Some learned embeddings



- A 2d projection of some learned vector-space embeddings from word2vec (for visualization, actual embeddings are much higher dimension)

Mikolov et al., https://arxiv.org/abs/1310.4546

Learning goals: embeddings

- What is a vector-space embedding, and what is it used for?
- What makes a good embedding?
- What is a proxy task, and why do we use it? What are some examples of proxy tasks?
- What are multi-task training, pre-training, and fine-tuning?
- What are some classic autoencoder-based models for pre-training embeddings?